CHORD: 분석 리포트

게시용 버전 · 원문 https://arxiv.org/abs/2607.00033 · arXiv:2607.00033v2 · NVIDIA (Zhu, Liu, Jain 외, 교신 Yan Chang) · CoRL 2026 · 코드 nvidia-isaac/video_to_data @ 9cad120 (Apache-2.0) · 분석일 2026-11-05 · 논문 그림 © 저자 · arXiv 라이선스는 CC BY 4.0이지만 PDF에 "© 2026 NVIDIA. All rights reserved."가 함께 있어 게시본에서는 논문 그림을 뺌

핵심 요약

1. **무엇을 하는가:** 사람 접촉점·법선으로 마찰원뿔 wrench 행렬을 만들고, 512개 방향의 지지 함수로 로봇 접촉과 비교하는 보상을 RL에 넣는다. 시연 하나당 정책 하나를 학습한다.
2. **보상은 위치가 아니라 미는 방향을 본다:** 같은 접촉점에서 법선만 90° 틀면 위치 보상은 1.00, 코드 보상은 0.33이다. 반대로 법선이 같으면 2 cm 이동에도 0.94다. 위치 보상을 대체하기보다 다른 축을 보완한다. 힘의 크기는 보지 않는다.
3. **논문 식과 코드 식이 다르다:** Eq. 3(512방향 위반 합의 exp)은 법선 10°에서 0.004이고, 코드(방향별 exp 평균)는 0.95다. 논문 식대로 구현하면 보상이 거의 항상 0이다.
4. **"r_fc 최대화 = force closure"는 512개 표본에서 성립하지 않는다:** 축 둘레 토크를 못 만드는 두 점 집기도 r_fc = 1, 표본 ε = 0.053이다. App. D의 ε 통계는 상한이다.
5. **헤드라인 수치는 맞지만 읽는 법이 필요하다:** 90.77%는 17개 가중 평균과 정확히 맞는다. 82.12%는 Fig. 5 막대의 가중 평균(0.781)이 아니라 과제 성공 비율이고, 1,831로는 정수가 안 된다. Table 1에서 SPIDER 대비 SP-SR 차이는 유의하지 않다(t = 0.58).
6. **FlashDexRetarget과의 정합:** FDR이 돌린 것은 PPO 공개 경로(19.7억 전이, 모션당 57-66 RTX 3090 GPU-h)이고, CHORD v2가 주장하는 것은 FlashSAC 경로(2.5억 전이, 과제당 약 2 L40S-h, VOC 일정 정확히 1/10)다. 같은 FlashSAC끼리면 격차는 2-3배 수준이다. 다만 82.12%와 그림은 v1과 바이트 단위로 같아서, 대규모 결과가 어느 레시피에서 나왔는지는 밝혀져 있지 않다.
7. **구현 가능성:** 코드·데이터는 공개되어 있지만 공식 학습은 NVIDIA GPU가 필요하다(FlashCHORD는 48 GiB). 이 맥에서는 불가. 핵심 보상은 numpy 80줄로 옮겼고 어느 시뮬레이터에도 붙일 수 있다.

01. 논문 분석: CHORD (arXiv:2607.00033)

- 제목: *Learning Dexterous Manipulation Using Contact Wrench Guidance From Human Demonstration* (CHORD: Contact Wrench Guidance from Human Demonstration in Robotic Dexterous Manipulation)
- 저자: Xinghao Zhu*ǂ, Zixi Liu*, Shalin Jain*, Chenran Li†, Milad Noori†, Michael Andres Lin† 외 15명, Yan Changǂ (NVIDIA). * 공동 1저자, † core contributor, ǂ 프로젝트 리드·교신저자
- 공개: arXiv v1 2026-06-22, **v2 2026-08-14 (분석 대상)** · 프로젝트 페이지 https://nvidia-isaac.github.io/video_to_data/chord/ (CoRL 2026으로 표기) · 코드 https://github.com/nvidia-isaac/video_to_data 의 `robotic_grounding/` (확인 커밋 `9cad120`, 2026-11-02)
- 라이선스: arXiv abs 페이지는 **CC BY 4.0**입니다. PDF 첫 쪽 아래에는 "© 2026 NVIDIA. All rights reserved."가 있습니다. 표기가 충돌하므로 thinkworld와 같이 보수적으로 판단해 **논문 그림은 개인용 리포트·PPT에만** 넣었습니다.
- 원문 대조: 프로젝트 페이지의 `chord.pdf`(42.1 MB)와 arXiv v2(41.8 MB)는 단어 12,075개 중 머리글 날짜(2026-8-14 vs 2026-8-17)와 arXiv 세로 도장만 다릅니다. 본문은 같습니다.
- 분석 범위: 본문 12쪽, 부록 A-G(24쪽까지), 프로젝트 페이지, 공식 코드(Isaac Lab PPO 경로와 FlashCHORD 경로). v1 ↔ v2 차이는 §5에 정리했습니다.

---

1. Abstract (한 줄 요약과 주장)

한 줄 요약: 사람 시연의 접촉을 **"물체에 줄 수 있는 힘·토크의 집합(wrench space)"**으로 바꾸고, 로봇 접촉이 만드는 집합과 512개 방향의 지지값(support function)으로 비교하는 보상을 만들어, 시연 하나당 정책 하나를 RL로 학습합니다. 접촉 위치가 같아도 미는 방향이 다르면 다른 보상을 받습니다.

논문이 내세우는 기여는 세 가지입니다(Sec. 1).

1. **CHORD:** 물체 중심 접촉 wrench 지도 보상과, 이를 받치는 학습 장치(레퍼런스 위 임의 시점 리셋, 물체 보조 VOC 커리큘럼, 사람 wrench에서 뽑은 교란, 잔차 행동)입니다.
2. **벤치마크:** 모션캡처 데이터 7종과 사내 영상 복원으로 양손 조작 과제 4,739개를 Isaac Lab 환경으로 만들었습니다. 데이터는 Hugging Face에 CC BY 4.0으로 공개되었습니다.
3. **검증:** 1,831개 과제 평균 성공률 82.12%, 전신(G1 + Dex3) 90.77%, 실물(Dexmate + Sharpa 양손) open-loop·closed-loop 전이를 보고합니다.

---

2. Method

2.1 전체 구조 (Fig. 2, Sec. 3)

```
사람 시연 τ_ref = {x_human_t, x_object_t}  (MANO 손 + 물체 부품 K개의 SE(3) 자세)
   │  QP 미분 IK로 로봇 관절 레퍼런스 x_robot_t (App. D, 프레임당 최대 200회 반복)
   │  프레임마다 손–부품 접촉점 p, 법선 n 추출 → 사람 wrench 행렬 𝒲_h,k (Eq. 1)
   ▼
시퀀스 하나 = 정책 하나 (Isaac Lab, Sharpa Wave 양손, 손목 6-DoF 부유)
   π(a | o_robot, o_object, x_robot_ref, x_object_ref)   잔차 행동: 레퍼런스 + Δ
   보상 r = r_task(+ r_relative) + r_imit + r_contact
            r_contact = r_cws(Eq. 3) − r_unintend − r_miss
   학습 장치: 임의 시점 리셋 + VOC 20스텝 고정 → 커리큘럼 감쇠, 사람 𝒲에서 뽑은 교란 wrench
   ▼
평가: 물체 15 cm / 40° 이탈 시 실패, 완료율 > 0.7이면 과제 성공 (Sec. 4.1)
```

2.2 과제·모방 보상 (Sec. 3.1, App. B)

- r_task = exp(−Σ_k ‖ x_obj,k ⊖ s_obj,k ‖² / var_obj) + r_relative. ⊖는 SE(3) 자세 차이입니다.
- r_relative = m(t)·exp(−e_p·e_R / var_rel)은 두 물체의 상대 자세 보상입니다. 삽입·붓기·뜨기처럼 물체끼리의 기하가 중요한 과제를 위해 넣었습니다. m(t)는 **시연의** 물체 간 거리로만 켜서, 정책이 물체를 떨어뜨려 보상을 피하지 못하게 했습니다 (App. B "Relative Task Reward"). 곱 형태라 위치와 회전이 둘 다 맞아야 보상이 높습니다.
- r_imit = exp(− ‖ x_robot − s_robot ‖² / var_imit)는 리타겟된 사람 동작 쪽으로 정규화합니다.

2.3 접촉 wrench 행렬과 지지 함수 (Eq. 1-2)

- 부품 k의 사람 접촉점 c개에서 마찰원뿔을 **단위 크기 모서리 d개**의 다면체로 근사합니다. 모서리 힘 f마다 primitive wrench w = [f, p × f] ∈ R^6를 만들고, 모두 모아 $\mathcal{W}$_h,k ∈ R^(6×cd)를 만듭니다(Eq. 1).
- 사람 손과 로봇 손은 접촉 수와 열 순서가 달라 행렬을 직접 비교할 수 없습니다. 그래서 미리 뽑은 단위 방향 b개 $\mathcal{B}$ ∈ R^(6×b)에 대해 **지지 함수 σ = max_col($\mathcal{B}^T\mathcal{W}$) ∈ R^b**를 계산해 비교합니다(Eq. 2). σ의 각 칸은 "그 방향으로 이 접촉들이 낼 수 있는 최대 wrench 성분"입니다.
- 논문은 μ, d, b를 적지 않았습니다. 코드는 μ = 0.1, d = 8(+ 법선 1개), b = 512, 토크는 물체 경계 구 반지름으로 나눕니다 (`04` D4).

2.4 접촉 wrench-space 보상과 벌칙 (Eq. 3)

$$r^k_{cws} = \exp\Big(-\frac{|\max(0,(1-\beta)\sigma_{h,k}-\sigma_{r,k})|^2}{v_{cws}} - \frac{|\max(0,\sigma_{r,k}-(1+\beta)\sigma_{h,k})|^2}{v_{cws}}\Big)$$

- 로봇 지지값이 사람 지지값의 [(1−β), (1+β)] 띠 안에 있으면 벌점이 없습니다. 아래로 벗어나면 "덜 미는 것", 위로 벗어나면 "필요 이상으로 미는 것"입니다.
- exp 커널은 항상 양수라서 접촉 불일치 벌칙을 따로 둡니다. 사람 지지값이 0인데 로봇이 양수이면 r_unintend, 반대면 r_miss입니다.
- v2에 추가된 단서(Sec. 3.1 끝): **물리적으로 정확한 다중 접촉 순 wrench 다면체를 복원하지 않습니다.** 접촉별 방향 능력을 정의하고 지지값을 비교하는 "지도 신호"이며, 실제 구동 가능성은 시뮬레이터가 보장합니다.
- 오른쪽 그림의 상자 뚜껑 예시가 동기입니다. 위치 기반 보상으로 최적화한 접촉은 사람 엄지와 가깝지만 법선이 거의 수직이라 뚜껑을 들지 못합니다. wrench 보상은 같은 물체 운동을 만드는 접촉을 고릅니다. 이 예시를 toy로 다시 만들었습니다 (`03` §2).

2.5 학습 장치 (Sec. 3.2, App. B)

장치	내용	출처
임의 시점 리셋	레퍼런스 위 무작위 프레임에서 시작. 손가락은 레퍼런스 자세와 펼친 자세 사이를 [0, 0.7]로 보간	App. B Termination and Reset
VOC 안정화 창	리셋 직후 VOC 배율 1.0으로 20스텝 유지, 그 뒤 커리큘럼 값	App. B
VOC 커리큘럼	물체마다 정책 행동 차원이 없는 가상 물체 제어 항이 PD wrench를 걸고, 커리큘럼으로 줄임. DexMachina[1]의 장치	Sec. 3.2
교란	사람 접촉 wrench 행렬 $\mathcal{W}$_h,k에서 뽑은 wrench로 물체를 밀어, 시연한 접촉 역학과 맞는 교란을 줌	Sec. 3.2
잔차 행동	손목 위치 0.05 m, 손목 회전 0.15 rad, 손가락 관절 0.15 rad 배율, EMA 0.3. 손목은 힘/토크 추종 제어기	App. B Action Space, ManipTrans[2] 방식
관측	손목 자세·속도, 손가락 관절, 모든 물체 자세, 손목·손가락·물체 명령 delta, 이전 행동, **손목 좌표계의 접촉 위치·힘 방향**	App. B Observation Space
종료	시간 초과, 손목 이탈, 물체 위치·회전 이탈	App. B

관측 잡음은 대규모 실험에서는 끄고, 실물 closed-loop 정책 학습에서만 켰다고 적었습니다.

학습 설정 (App. B "Training Details", v2에서 추가): FlashSAC, 환경 2,048개, 시퀀스별 정책을 L40S 한 장에서 약 2시간에 학습합니다. "과제 간 일반화가 아니라 학습 레시피의 확장성"을 보인 것이라고 스스로 한정했습니다. 비교로 DexMachina, ManipTrans, Spider는 과제당 약 10, 20, 4시간이 걸린다고 적었습니다. 이 단락은 v1에 없습니다(§5).

2.6 잡음 있는 시연: 축소 force-closure 보상 (Sec. 3.2)

영상 복원처럼 손-물체 정렬이 부정확하면 사람 wrench를 그대로 맞추는 것이 위험합니다. 이때는 r_fc = (1/B) Σ_b 1[σ_r,k,b > ε]로 바꿉니다. 로봇 지지값이 모든 기저 방향에서 양수가 되도록 장려하고, "최대화하면 force closure와 같다"고 적었습니다. 이 동치는 방향이 촘촘할 때만 성립합니다. 512개 무작위 방향에서는 force closure가 아닌 두 점 집기도 r_fc = 1이 됩니다(`03` §4).

2.7 전신 확장 (Sec. 3.2, App. E)

- **손만 있는 시연(자기중심 영상):** 손목 궤적에서 전신 동작을 만드는 inpainting 모듈을 MotionBricks[37] 구조로 BONES Seed 데이터에 학습합니다. 그 전신 레퍼런스를 SONIC[20] 전신 제어기로 추종하고, CHORD가 그 위에 잔차를 배웁니다(비손가락 관절 배율 0.5, 손가락 0.15).
- **전신 시연(3인칭 영상, TPV):** 전신은 있지만 손가락 복원이 부정확해서[38] r_fc를 씁니다.
- 리셋 때 G1 어깨 yaw 관절을 0.5만큼 벌려 팔을 물체에서 떼어 두고, 정책이 물체 쪽으로 회복하게 합니다. 전신 관측에서는 접촉 관측을 뺐습니다.

2.8 벤치마크 구성 (Sec. 3.3, App. D)

- **원천:** ARCTIC, TACO, HOT3D, OakInk2, DexYCB, GRAB, H2O의 MANO 손과 물체 6-DoF 궤적을 통일된 형식(손목 자세, MANO 관절 21개, 링크별 접촉, 물체 자세)으로 변환합니다.
- **HOT3D 분할:** 긴 자기중심 녹화를 손끝-물체 거리 2 cm 기준으로 접촉 구간을 나누고, 15프레임 이하의 끊김은 잇고, 프레임별 다수결로 주 물체를 정합니다. 약 294개 녹화가 약 4,045개 클립이 됩니다. **4,739개 중 약 85%가 HOT3D**입니다(`04` C1).

- **품질 검사:** 손을 캡슐로 모델링해 손-물체(볼록 껍질 기준), 손-손 침투가 어느 프레임에서든 2 cm를 넘으면 버립니다. 볼록 껍질/메쉬 부피비가 3을 넘는 오목한 물체(머그, AR 안경)는 손-물체 검사에서 뺍니다. Isaac Lab에서 300스텝 재생이 끝나고 영상이 저장되어야 통과합니다.
- **통계(Fig. 3):** 시간 길이, 접촉 이벤트 수, Ferrari-Canny ε를 DexMachina·ManipTrans·Spider 과제 묶음과 비교합니다. ε는 512개 기저 방향 지지값의 최솟값으로 계산합니다. 이 값은 진짜 ε의 상한입니다(`03` §4).

---

3. Experiments

3.1 평가 설정 (Sec. 4.1)

- 로봇은 Sharpa Wave 양손이고 시뮬레이터는 Isaac Lab입니다. 모든 과제에서 VOC 이득, 커리큘럼, 보상 가중치를 같게 썼습니다.
- rollout이 물체 위치 15 cm 또는 회전 40°를 넘지 않고 끝나면 성공입니다. 완료율(성공 rollout / 전체 rollout)이 0.7을 넘으면 과제 성공으로 셉니다. 0.7은 경험적 기준이라 세밀한 차이를 다 담지 못한다고 v2에서 덧붙였습니다.
- 코드의 종료 임계값은 논문과 다릅니다(PPO 경로 물체 0.2 m / 0.7 rad, FlashCHORD 경로 물체 0.10 m / 0.50 rad, 손목 0.15 m). 완료율 0.7 규칙은 공개 코드에서 찾지 못했습니다(`04` D3).

3.2 1,831개 대규모 평가 (Fig. 5 왼쪽)

데이터	종류	n	성공률 (그림 숫자)
HOT3D-1Obj	강체	1,131	0.772 ± 0.012
OakInk2-1Obj	강체	147	0.786 ± 0.132
OakInk2-2Obj	강체	130	0.746 ± 0.038
ARCTIC	관절	196	0.814 ± 0.028
TACO	다물체	77	0.935 ± 0.028
HOT3D-2Obj	다물체	150	0.753 ± 0.035

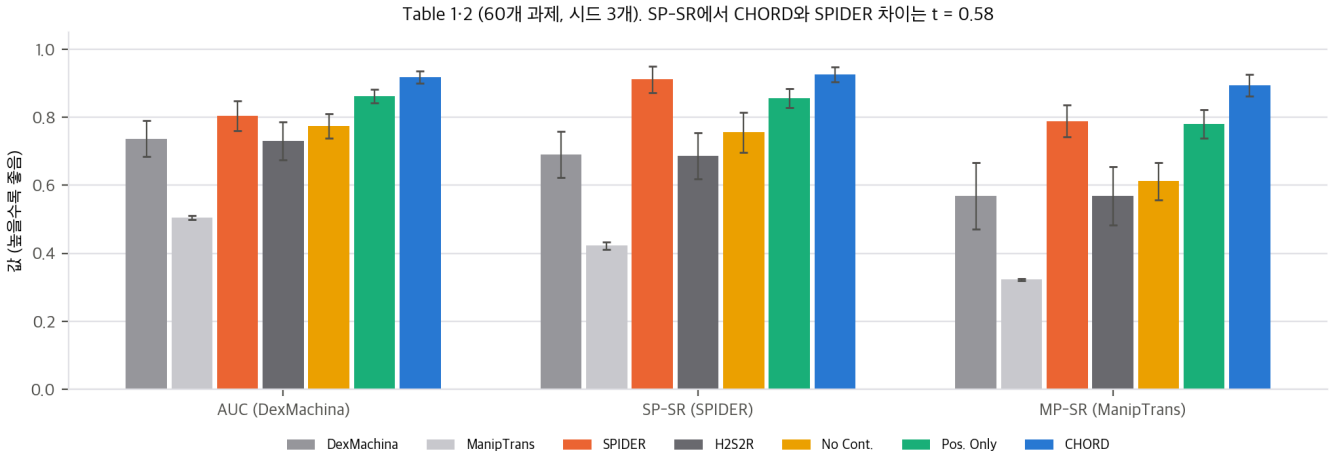
- n의 합은 정확히 1,831입니다. **n 가중 평균은 0.781**이고 단순 평균은 0.801입니다. 둘 다 초록의 82.12%가 아닙니다. 막대는 완료율 평균이고, 82.12%는 "완료율 > 0.7인 과제 비율"로 읽어야 맞습니다. 그런데 82.12% × 1,831 = 1,503.6이라 과제 수로 나누어떨어지지 않습니다. 시드 3개(5,493)로 보면 4,511/5,493 = 82.123%가 맞습니다. 집계 방식은 논문에 없습니다(`04` C1).
- 평가 과제의 70%(1,281개)가 HOT3D입니다.
- 어려운 과제일수록 VOC를 줄이기 전에 접촉 wrench 보상이 먼저 수렴해야 안정적이라고 적었습니다.

3.3 기준선 비교 (Table 1, v2에서 새로 만듦)

60개 과제(강체 17, 관절 24, 다물체 19), 시드 3개입니다. 기준선은 모두 Sharpa 손으로 다시 구현했습니다(App. G).

방법	AUC ↑	SP-SR ↑	MP-SR ↑	JPE (°) ↓	RPE (cm) ↓	MPPE (cm) ↓
DexMachina	0.737 ± 0.053	0.690 ± 0.068	0.569 ± 0.097	13.10 ± 2.31	9.92 ± 1.64	15.88 ± 0.70
ManipTrans	0.506 ± 0.006	0.423 ± 0.011	0.323 ± 0.003	30.07 ± 0.61	9.91 ± 0.42	21.24 ± 6.19
SPIDER	0.804 ± 0.044	0.911 ± 0.039	0.789 ± 0.047	41.35 ± 2.44	6.99 ± 1.56	14.28 ± 2.79
Human2Sim2Robot	0.730 ± 0.056	0.686 ± 0.068	0.569 ± 0.085	11.80 ± 1.50	9.78 ± 1.03	16.59 ± 0.51

방법	AUC ↑	SP-SR ↑	MP-SR ↑	JPE (°) ↓	RPE (cm) ↓	MPPE (cm) ↓
CHORD	0.918 ± 0.018	0.926 ± 0.022	0.894 ± 0.032	4.34 ± 0.25	5.14 ± 0.54	5.11 ± 0.39



해석 (재계산은 `04` C3):

- **SPIDER 지표에서는 사실상 동률입니다.** SP-SR 0.926 vs 0.911, Welch t = 0.58(p = 0.60, 시드 3개)입니다. 차이가 뚜렷한 것은 AUC(t = 4.2), MP-SR(t = 3.2), 관절 오차 JPE(t = –8.5), MPPE(t = –5.6)입니다. SP-SR은 임계가 MP-SR보다 훨씬 느슨해서(시간 평균 위치 오차 10 cm·자세 0.5 rad 대 3 cm·30°) 두 방법 모두 0.9를 넘고, 그래서 차이가 묻힌다고 보는 편이 맞습니다. (2026-11-12 수정: 처음에는 SP-SR이 위치를 궤적 평균으로 중심화한다고 적었습니다. 그것은 SPIDER 원 스크립트의 양손 모드 동작입니다. CHORD 공개 평가 코드는 중심화하지 않은 오차를 쓰고, Table 1이 어느 정의로 계산됐는지는 논문에 없습니다. `04` D13)
- SPIDER는 물체는 잘 따라가지만 관절 오차가 41°로 가장 큽니다. 샘플링 최적화가 사람 손 모양과 무관한 자세를 찾는다는 뜻입니다. CHORD는 4.3°입니다.
- 기준선 재현은 공정성을 신경 썼습니다. DexMachina는 변형 4개 × 시드 5개 중 가장 좋은 것(원래 이득 + hybrid 행동)을, ManipTrans는 Inspire 손에서 원 논문 경향(한 손 64.1%, 양손 29.7%)을 먼저 재현한 뒤 Sharpa로 옮겼습니다. SPIDER는 CHORD의 IK를 썼습니다. 공동 1저자 Zixi Liu는 SPIDER의 공저자이기도 합니다.
- 다만 App. G는 DexMachina와 SPIDER를 **시드 5개**로 돌렸다고 쓰고, Table 1 캡션은 시드 3개라고 씁니다. 어느 쪽이 표에 들어갔는지 확인할 수 없습니다.

3.4 접촉 지도 ablation (Table 2)

변형	AUC	SP-SR	MP-SR	JPE	RPE	MPPE
CHORD (wrench)	0.918	0.926	0.894	4.34	5.14	5.11
Position Only (DexMachina식 위치 보상)	0.862 ± 0.019	0.856 ± 0.028	0.780 ± 0.042	6.41 ± 0.44	7.81 ± 0.64	7.63 ± 0.06
No Contact	0.774 ± 0.036	0.756 ± 0.059	0.612 ± 0.055	9.81 ± 0.73	10.91 ± 1.24	14.38 ± 2.23

- 세 변형은 접촉 보상 형태만 다르고 나머지 보상은 같습니다. CHORD 행은 Table 1과 숫자가 같으므로 같은 실행을 다시 쓴 것입니다.
- SP-SR 기준으로 접촉 보상이 없을 때 대비 이득 0.170 중 위치 보상이 0.100, wrench로 바꾼 몫이 0.070(41%)입니다. Position Only 대비 모든 지표에서 $p < 0.03$입니다(Welch, 시드 3개).
- 캡션은 "시드당 환경 4,096개"라고 씁니다. App. B의 FlashSAC 2,048개와 다릅니다(`04` D6).

3.5 보상-성공 상관 (Fig. 6)

1,831개 실행에서 정규화한 CWS 보상과 완료율의 Pearson r = 0.798(데이터별 0.76-0.89)입니다. r^2 = 0.637이라 "분산의 약 2/3"와 맞습니다. 관계가 단조지만 포화형이라 직선 적합(y = 1.112x + 0.057)은 높은 보상 구간을 과소평가한다고 스스로 적었습니다. 직선은 x = 0.85에서 1에 닿습니다. 이 상관은 같은 정책이 보상과 성공을 함께 만든 결과라서, 보상이 원인이라는 증거는 아닙니다. 쉬운 과제가 둘 다 높을 수 있습니다.

3.6 긴 시퀀스 (Fig. 7)

DexMachina ADD-AUC로 시퀀스별 추적 정확도를 시간 길이에 따라 그렸습니다. CHORD는 40-48초 시퀀스에서도 대부분 0.85-0.98을 유지하고, DexMachina와 ManipTrans는 길이가 늘수록 떨어집니다. 시드 최대 5개의 평균입니다. 코드의 `motion_speed`가 0.5라서 정책은 시연을 절반 속도로 재생합니다(`04` D9). 그림의 초가 원본 시간인지 재생 시간인지는 적혀 있지 않습니다.

3.7 전신 (Fig. 5 오른쪽, Table 3, Fig. 8)

범주	n	CHORD	Position Only
강체	4	0.994 ± 0.008	0.460 ± 0.487
관절	4	0.914 ± 0.107	0.000 ± 0.000
다물체	4	0.866 ± 0.129	0.192 ± 0.166
전체	12	0.925 ± 0.104	0.217 ± 0.333

- 초록의 90.77%는 손만 있는 12개와 TPV 5개(0.867)를 합친 17개의 가중 평균과 정확히 맞습니다. Table 3은 TPV를 뺀 12개입니다(`04` C2).
- Position Only가 무너지는 폭(0.925 → 0.217)이 손 시뮬레이션(0.926 → 0.856)보다 훨씬 큽니다. 다섯 손가락 사람 손 → 세 손가락 Dex3처럼 형태가 다르면 위치 대응이 성립하지 않고, wrench는 "물체에 미치는 효과"로 비교하므로 형태 차이를 넘는다는 해석입니다. 저자 주장의 가장 강한 근거입니다. 다만 과제당 4개, 시드 4개라 표본이 작습니다.
- Fig. 5와 Table 3은 같은 과제인데 표준편차가 다릅니다(강체 0.038 vs 0.008). 하나는 과제 간, 하나는 시드 간일 가능성이 큽니다.

3.8 실물 (Sec. 4.6, App. F)

- Dexmate 로봇 + Sharpa 양손, Vicon으로 로봇과 물체 자세를 추적합니다. 61차원 관절 벡터를 20 Hz ONNX 정책으로 냅니다.
- open-loop 행동 청크 실행: 그릇 건네기 5/5, 손 안 접시 회전 3/5, 그릇-접시 쌓기 5/5, 믹서 열고 닫기 5/5, 캡슐 머신 집어 놓기 5/5. 합계 23/25(Wilson 95% [0.75, 0.98])입니다.
- 상자 집어 놓기는 open-loop 6/7, closed-loop 7/7입니다(Fisher p = 1). closed-loop 정책도 고정된 시연 레퍼런스를 따라가며 다시 계획하지 않는다고 분명히 했습니다.
- 실패 원인은 초기 물체 배치와 접촉 전환에서의 sim-to-real 차이입니다.

3.9 원격조종 비교 (Sec. 4.7)

같은 과제를 원격조종으로 재현하는 소규모 정성 연구입니다. 상자 들기가 특히 어려웠고, 원인으로 IK가 손가락 배치를 보존하지 못함, 촉각·접촉 피드백 부재, 관절 토크를 직접 제어할 수 없음을 들었습니다. 믹서와 와플 기계는 연습하면 물체 궤적은 비슷해지지만 접촉이 달라 쥐는 힘이 약했다고 합니다. 수치는 없습니다.

3.10 부록 실험

- **시연 잡음 (App. A, Fig. 10):** 손목 위치 10-60 mm, 회전 5-30°, 접촉 위치 5-30 mm, 법선 5-30°의 random walk 잡음 6단계(Table 4). AUC가 1단계에서도 0.5-0.6이라 Table 1의 0.918보다 훨씬 낮습니다. 과제 묶음이 적혀 있지 않고, 잡음 없는 기준점도 없습니다. 그림에서 읽으면 손 잡음만 있을 때 0.60 → 0.54, 손+접촉 잡음 0.61 → 0.33(5단계) → 0.48, force-closure 보상 0.52 → 0.50으로 띠가 크게 겹칩니다. "높은 잡음에서는 force closure가 낫지만 정교함이 줄어든다"는 결론은 경향 수준입니다.
- **사람 영상 끝단 검증 (App. C):** 단안 RGB + 물체 이름으로 MoGe(깊이), Grounding DINO·SAM2·SAM3D(물체), FoundationPose(추적), ViPE + Dyn-HaMR(손), Gaussian splat + DROID-SLAM(공동 정제)을 거쳐 붓 과제 하나를 복원하고 AUC 0.647을 얻었습니다. 과제 하나의 결과입니다.

---

4. Limitations

4.1 저자가 밝힌 한계 (Sec. 6)

1. 실물 배포가 상태 기반 관측(Vicon)에 의존합니다.
2. 접촉 지도에는 비교적 깨끗한 시연이 필요합니다. 잡음이 크면 force closure 가정에 기대야 합니다.
3. 물체 자세 오차는 불완전한 평가 지표입니다. 어떤 과제는 정확한 배치가 필요 없고, 어떤 과제는 작은 오차로도 기능이 실패합니다.

4.2 분석하며 확인한 한계

1. **대규모 수치와 학습 레시피의 시점이 어긋납니다.** 82.12%, 90.77%와 Fig. 5-9의 그림 파일은 v1과 v2가 바이트 단위로 같습니다. FlashSAC로 2시간에 학습한다는 단락은 v2에만 있고, FlashSAC 코드(FlashCHORD)는 2026-09-23에야 공개되었습니다. 1,831개 평가가 어느 레시피로 학습되었는지 논문은 밝히지 않습니다(`04` D2).
2. **Eq. 3과 코드 보상이 다른 함수입니다.** 논문은 512방향 오차 제곱합의 exp이고, 코드는 방향별 exp의 평균이며 양쪽 지지값이 모두 양수인 칸만 셉니다. toy에서 Eq. 3은 법선 오차 10°에서 0이 되지만 코드 보상은 0.95입니다(`03` §2).
3. **"force closure와 동치"는 512개 무작위 방향에서 성립하지 않습니다.** 두 점 집기는 축 둘레 토크를 못 만들어 force closure가 아닌데, 표본 최솟값은 0.053이고 r_fc = 1입니다(`03` §4). App. D의 ε 통계도 상한입니다.
4. **wrench 보상은 힘의 크기를 보지 않습니다.** 코드는 접촉력의 방향만 쓰고 모서리를 단위 길이로 만듭니다. "힘·토크로 비교한다"는 설명과 달리, 실제로는 접촉 위치와 법선(방향 능력)만 비교합니다(`03` §3, `04` D7).
5. **위치 오차에는 오히려 둔감합니다.** 법선이 맞으면 접촉점이 2 cm 밀려도 코드 보상은 0.94입니다(위치 보상은 0.37). 토크를 경계 구 반지름으로 나누기 때문입니다. wrench 보상은 위치 보상보다 "엄격한" 것이 아니라 "다른 축을 보는" 보상입니다.
6. **SPIDER와의 차이는 SPIDER 지표에서 유의하지 않습니다**(SP-SR t = 0.58).
7. **설정값이 논문과 코드에서 다릅니다.** 종료 임계값, 환경 수(2,048 vs 4,096), 시드 수(3 vs 5), μ = 0.1 미기재, 시연 절반 속도 재생(`04` D3-D9).
8. **시퀀스 하나에 정책 하나입니다.** 저자도 "과제 간 일반화가 아니다"라고 했습니다. 4,739개를 다 쓰려면 4,739번 학습해야 합니다. FlashDexRetarget이 정확히 이 지점을 공략했습니다(`02` §3).
9. **평가가 HOT3D에 쏠려 있습니다.** 1,831개 중 70%, 4,739개 중 85%입니다. HOT3D 분할 클립은 짧은 원자 상호작용이 많습니다.
10. **실물 표본이 작습니다.** 과제당 5회, 상자 open/closed-loop 비교는 7회씩입니다.

---

5. v1 ↔ v2 차이

항목	v1 (2026-06-22)	v2 (2026-08-14)
Table 1	기준선마다 자기 과제 묶음·자기 지표. DM 묶음 AUC 0.232 vs 0.687, MT 묶음 MT-SR 0.428 vs 0.639, SP 묶음 SP-SR 0.333 ± 0.488 vs 0.359 ± 0.482, 자체 9개 과제(Ours-1·2)에서 DM AUC 0.211 vs 0.895, SP-SR 0.133 vs 0.999, 0.533 vs 0.982	60개 과제 × 시드 3개, 지표 6개를 모든 방법에 같이 적용. Human2Sim2Robot 추가
Table 2	ARCTIC 두 시퀀스: box grab 0.702 / 0.334 / 0.384, mixer use 0.894 / 0.624 / 0.423 (CHORD / 위치 / 없음)	60개 과제, 지표 6개
학습 설정	없음	"FlashSAC, 환경 2,048개, L40S 한 장 약 2시간", 기준선 10/20/4시간
방법 단서	없음	"정확한 순 wrench 다면체를 복원하지 않는다"
평가 단서	0.7 기준만	0.7이 세밀한 차이를 담지 못한다는 단서, "Evaluation Tasks" 단락
실물	정성 서술 위주	과제별 시도 수(5/5, 3/5 …), 상자 6/7 vs 7/7
그 밖	—	참고문헌을 등장 순서로 재번호, 저자 목록 조정 (Mrinal Verghese 추가, Michael Andres Lin †), H2S2R 인용 [43] 추가
변하지 않은 것	초록 수치 82.12%, 90.77%, 4,739, 1,831	같음. **본문 2-12쪽의 래스터 그림 11개가 바이트 단위로 같음**(Fig. 2, 4-9)

v1의 Table 1은 SP 묶음에서 0.359 vs 0.333(표준편차 0.48)처럼 사실상 차이가 없는 비교를 "matching or outperforming"으로 묶었습니다. v2는 비교를 통일해 훨씬 해석 가능해졌습니다. 다만 그림과 초록 수치가 그대로라서, v2의 새 학습 설정 단락이 대규모 결과에도 적용되는지는 알 수 없습니다.

03. 유도와 재계산: 지지 함수, 보상 식, force closure, 예산

수치는 `analysis/wrench_toy.py` (→ `wrench_toy_results.json`)와 `analysis/table_check.py` (→ `table_check_results.json`)가 만듭니다. 둘 다 numpy·scipy, 단일 스레드, 1초 안에 끝납니다. toy는 공식 코드 `video_to_data @ 9cad120`의 함수를 그대로 옮겼습니다(`04` §1).

1. 지지 함수가 비교하는 것

wrench 행렬 $\mathcal{W}$의 열 w_j는 접촉 하나의 마찰원뿔 모서리 하나가 만드는 [f, p × f / r_c]입니다(코드는 토크를 물체 경계 구 반지름 r_c로 나눔). 지지 함수 σ(b) = max_j $b^{\top}$w_j는 **열들의 볼록 껍질 conv($\mathcal{W}$)의 지지 함수**와 같습니다. 여기서 몇 가지가 바로 나옵니다.

1. **열 순서와 개수에 무관합니다.** 사람 손(접촉 c_h개)과 로봇 손(c_r개)을 같은 길이 b의 벡터로 비교할 수 있는 이유입니다. 논문이 지지 함수를 쓴 동기(Sec. 3.1)와 맞습니다.
2. **접촉을 더하면 지지값은 줄지 않습니다.** max에 열이 추가될 뿐입니다. 그래서 로봇이 사람보다 많이 닿으면 σ_r > (1 + β)σ_h인 방향이 생기고, too_large 항이 이를 깎습니다(§2.4).
3. **0 아래는 잘립니다**(코드 `clamp(min=0)`). 원점이 conv($\mathcal{W}$) 밖에 있으면 일부 방향의 지지값이 음수인데, 이를 0으로 만들어 "그 방향으로는 못 민다"로 통일합니다.
4. **크기가 없습니다.** 열은 단위 모서리이므로 σ는 "어느 방향으로 밀 수 있는가"의 기하만 담습니다. 힘의 크기는 들어가지 않습니다(§3).
5. **원점이 conv($\mathcal{W}$) 내부에 있으면 force closure이고**, 그때 모든 방향의 σ는 양수입니다. min_b σ(b)는 원점에서 껍질 면까지의 최단 거리, 즉 Ferrari-Canny ε입니다. 단, **모든** 단위 방향에 대한 최솟값일 때만 그렇습니다(§4).

2. Eq. 3과 코드 보상

2.1 두 식

- **논문 Eq. 3:** r = exp(−(‖ max(0, (1−β)σ_h − σ_r) ‖ ² + ‖ max(0, σ_r − (1+β)σ_h) ‖ ²) / v). 512개 방향의 위반을 **모두 더한 뒤** exp를 씌웁니다.
- **코드** `contact_wrench_support_reward_jit`: 방향마다 loss_b = (위반)²를 계산하고, r = (1/N_cmd) Σ_b 1[σ_h,b > 10^{-3} ∧ σ_r,b > 10^{-3}] · exp(−loss_b / v). **방향별 exp의 평균**이고, 로봇 지지값이 0인 방향은 0점, 사람 지지값이 0인 방향은 아예 세지 않습니다. 그 뒤 body 평균, 손 평균을 냅니다.

β = 0.1, v = 0.1(코드 기본값)에서 두 식은 전혀 다르게 움직입니다. Eq. 3은 지수 안의 합이 위반 방향 수에 비례해 커지므로, 몇 도만 틀려도 거의 0이 되는 사실상 이진 보상입니다. 코드 보상은 "띠 안에 들어온 방향의 비율"에 가까운 완만한 보상입니다.

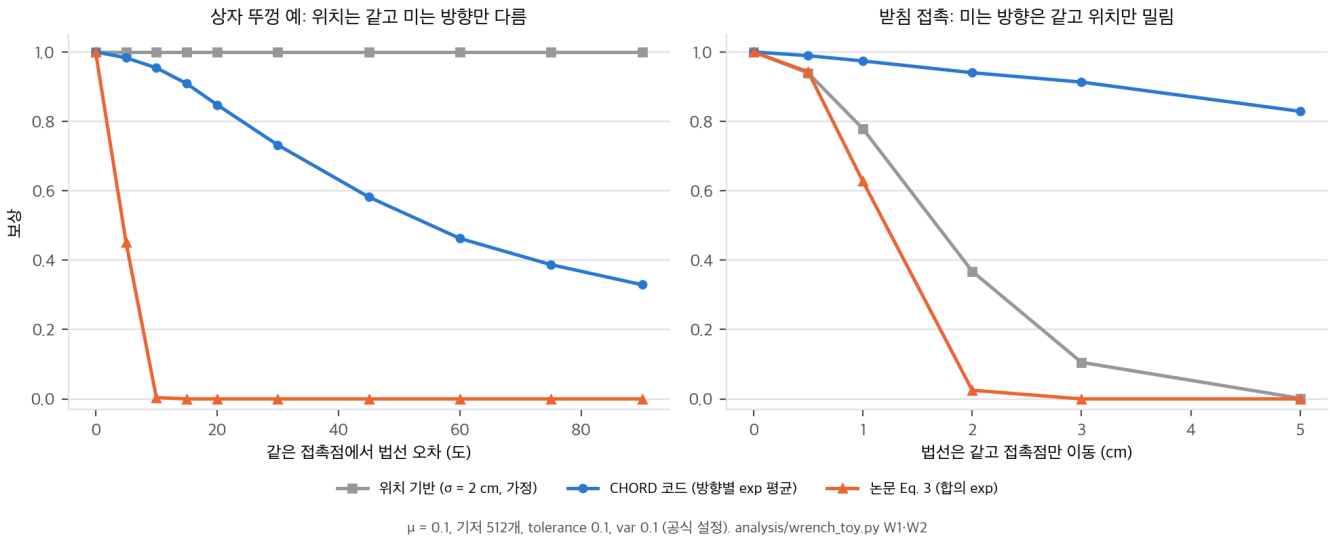
2.2 상자 뚜껑: 같은 위치, 법선만 회전 (W1)

Sec. 3.1 오른쪽 그림을 그대로 만들었습니다. 상자(반치수 10 × 7.5 × 5 cm, r_c = 13.5 cm)의 앞면 위쪽, 뚜껑 턱 아래에 사람 엄지가 위로 밉니다(법선 +z). 로봇은 같은 점에서 법선을 앞면 안쪽(−x) 방향으로 돌립니다.

법선 오차	위치 보상 (σ = 2 cm 가정)	코드 보상	Eq. 3	놓친 방향 비율
0°	1.00	1.00	1.00	0.00
5°	1.00	0.98	0.45	0.01
10°	1.00	0.95	0.004	0.03

법선 오차	위치 보상 (σ = 2 cm 가정)	코드 보상	Eq. 3	놓친 방향 비율
20°	1.00	0.85	0.00	0.08
45°	1.00	0.58	0.00	0.17
90°	1.00	0.33	0.00	0.42

- 위치 보상은 법선을 전혀 보지 못합니다. 논문이 말한 실패 방식이 정확히 재현됩니다.
- 코드 보상은 90°(완전히 다른 방향으로 미는 접촉)에서도 0.33을 줍니다. 단일 접촉의 지지값은 6차원 방향의 절반 남짓에서 양수이고, 그중 상당수는 법선을 바꿔도 비슷하기 때문입니다. 놓친 방향 비율 0.42는 r_miss(가중치 −1)로 따로 벌합니다.



2.3 법선은 같고 위치만 이동 (W2)

이동	위치 보상	코드 보상	Eq. 3
0.5 cm	0.94	0.99	0.94
1 cm	0.78	0.97	0.63
2 cm	0.37	0.94	0.03
5 cm	0.00	0.83	0.00

토크 성분이 p × f / r_c라서 1 cm 이동은 토크를 약 0.074만 바꿉니다. **코드 보상은 위치 오차에 위치 보상보다 훨씬 관대합니다.** wrench 보상은 위치 보상을 "더 엄격하게" 만든 것이 아니라, 법선에 민감하고 위치에 둔감한 **다른 축의 보상**입니다. 위치는 손 keypoint·관절 추적 보상과 물체 추적 보상이 따로 붙잡습니다. Table 2에서 Position Only가 위치 오차(RPE, MPPE)까지 CHORD보다 나쁜 것은, 위치 보상이 위치를 직접 맞춰도 물체가 맞게 움직이지 않으면 물체 오차가 커진다는 뜻으로 읽힙니다.

2.4 오차가 여러 방향에 퍼질 때와 추가 접촉 (W5, W8)

- 양쪽 면을 잡는 두 손가락 집기에서 두 법선을 같은 각도로 틀면, 2°에서 이미 15%의 방향이 띠를 벗어납니다. Eq. 3은 5°에서 0.07, 10°에서 0이고, 코드는 0.995, 0.96입니다. **논문 식 그대로 구현하면 학습 초기에 보상이 거의 항상 0이라 신호가 없었을 것입니다.** 코드가 방향별 평균으로 바꾼 것은 합리적이지만 논문에 적혀 있지 않습니다(`04` D1).
- 사람이 아랫면 하나만 받치는데 로봇이 옆면에도 닿으면, 사람 지지값이 0인 138개 방향(27%)에서 로봇 지지값이 생깁니다. 코드는 이 방향을 세지 않습니다. 사람 활성 방향에서 (1+β)배를 넘는 부분만 too_large로 깎여 0.87이 됩니다. Eq. 3은 0입니다. 코드의 r_unintend는 **사람 접촉이 없는 body**에만 걸리므로, 같은 물체 위의 추가 접촉은 약하게만 벌합니다.

2.5 마찰계수 (W4)

코드는 μ = 0.1(원뿔 반각 5.7°)입니다. 논문에는 값이 없습니다. μ를 1.0(45°)까지 올려도 법선 30° 오차에서 코드 보상은 0.73 → 0.84로 조금 관대해질 뿐입니다. 보상 모양은 μ보다 접촉 기하에 더 크게 좌우됩니다. μ = 0.1은 사실상 "법선 방향으로만 미는 접촉"을 가정한 것이라, 실제 마찰로 버티는 집기를 사람 쪽에서 과소평가합니다(내 판단).

3. 힘 크기 불변성 (W3)

로봇 쪽 "법선"은 시뮬레이터 접촉력의 방향입니다(`wrench_preprocess_jit`). `compute_wrench_space`가 이를 정규화하므로, 접촉력을 0.1배, 1배, 10배로 바꿔도 지지값 차이는 최대 1.1×10^{-16}입니다. 즉 CHORD 보상은 **접촉 위치와 미는 방향**만 비교합니다. 논문 초록의 "forces and torques they can induce"는 크기가 아니라 방향 능력입니다. 쥐는 힘이 약해 물체가 미끄러지는 실패는 물체 추적 보상과 종료 조건이 막아야 합니다. DexForce와의 차이(`02` §4)가 여기서 나옵니다.

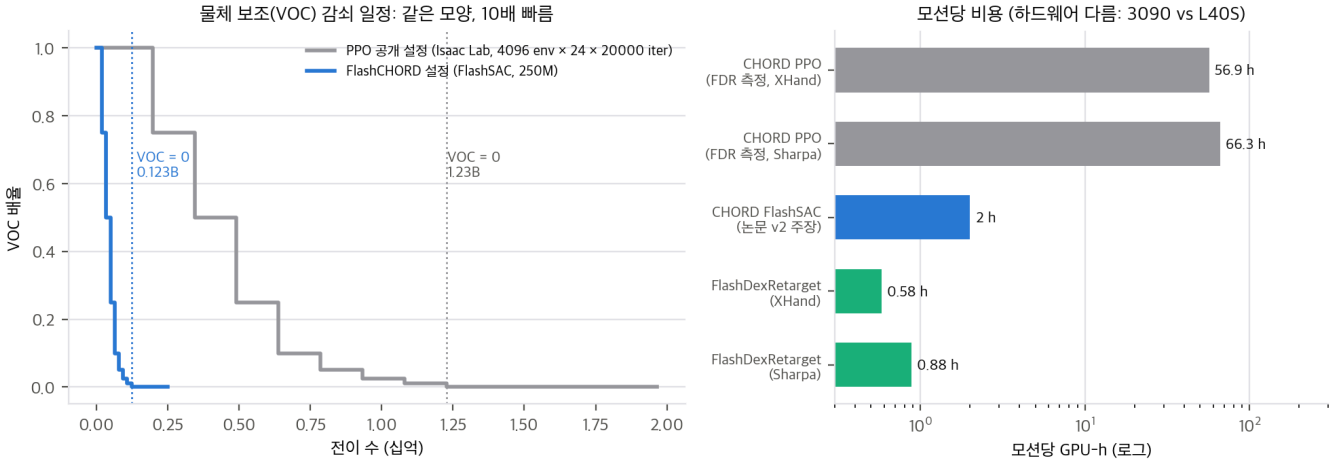
4. 512개 표본 방향과 force closure (W6, W9)

App. D는 Ferrari-Canny ε를 "512개 기저 방향 지지값의 최솟값"으로 계산하고, Sec. 3.2는 r_fc = (1/B) Σ 1[σ_b > ε]를 "최대화하면 force closure와 같다"고 썼습니다.

- **반례:** 마찰 있는 점접촉 두 개로 양쪽 면을 집으면(antipodal), 두 점을 잇는 x축 둘레 토크를 만들 수 없습니다(p × f의 x성분이 모든 열에서 정확히 0). 진짜 ε는 0이고 force closure가 아닙니다. 그런데 512개 무작위 방향의 최솟값은 **0.053**이고 **r_fc = 1.0**입니다. 순수 τ_x 방향에서 가장 가까운 표본 방향까지 15.8° 떨어져 있어서, 그 표본은 다른 성분 덕분에 양수 지지값을 얻습니다.
- **덮는 정도:** 6차원 단위 구에서 512개 무작위 방향은 임의의 방향에서 가장 가까운 표본까지 중앙값 22.1°, 95% 분위 29.8°, 최대 38°입니다(시험 방향 20,000개). 얇은 축 하나가 빠진 집합을 잡아내기에는 성깁니다.
- **결과:** App. D의 ε 통계(Fig. 3)는 진짜 ε의 상한이고, r_fc = 1은 force closure의 필요조건일 뿐입니다. μ = 0.1에서 면 중앙 4개 접촉의 표본 ε는 0.103, 6개는 0.157입니다. 표본 방향을 늘리거나, 방향 대신 LP로 원점 포함 여부를 직접 검사하면 고칠 수 있습니다.
- **코드의 r_fc는 손마다 따로입니다.** body 축을 amax로 합치고, 손 하나의 지지값만으로 비율을 계산합니다(`rewards.py: force_closure_reward`). 양손으로 잡아야 force closure가 되는 큰 물체는 손별 r_fc가 1에 닿지 못할 수 있습니다. 기본 가중치는 0(꺼짐)이고 TPV 전신 과제에서만 켭니다.
- 기저 표본 seed에 따른 흔들림은 작습니다. 같은 장면에서 seed 20개의 코드 보상 표준편차는 0.013(30° 법선 오차), 0.005(집기 10°)입니다(W7).

5. 학습 예산 산술 (C7)

항목	PPO 공개 경로 (Isaac Lab)	FlashCHORD 경로 (FlashSAC)
병렬 환경	4,096	4,096 (논문 App. B는 2,048)
총 전이	4,096 × 24 × 20,000 = 19.66억	2.50억
VOC = 0 시작	iteration 12,500 = 12.29억	1.229억 (PPO의 정확히 1/10)
물체 keypoint 가중치	0 → 0.1 → ⋯ → 1 → 20 (커리큘럼)	처음부터 20
마지막 단계	—	2.0억부터 리셋의 절반을 첫 프레임에서 시작
모션당 시간	FDR 측정 56.9 h(XHand), 66.3 h(Sharpa), RTX 3090	논문 약 2 h, L40S
역산 처리량	9.6k·8.2k 전이/s (20,000 iteration 가정)	34.7k 전이/s



- FlashCHORD 임계값 처음 9개는 PPO 일정 × 98,304 / 10과 정확히 같습니다(`table_check.py`의 `flash_first9_thresholds_equal_ppo_div10 = true`).
- 커리큘럼은 `bisect_right`라서 VOC가 0이 되는 것은 표의 9번째 값(14,000)이 아니라 8번째 값(12,500)부터입니다. 처음 정리할 때 14,000으로 잘못 읽었다가 코드를 다시 보고 고쳤습니다.
- FDR 50개 모션의 비교 함의는 `02` §3에 정리했습니다.

6. 표 수치 재계산 요약

#	대상	결과
C1	82.12% (1,831개)	Fig. 5 n 합 = 1,831 ✅. 그림 막대의 n 가중 평균은 0.781로 82.12%와 다름 → "과제 성공 비율"로 읽어야 함. 1,831로는 정수 불가, 시드 3개면 4,511/5,493 = 82.12%
C2	90.77% (전신)	Fig. 5 오른쪽 17개 가중 평균 = 0.9077 ✅. Table 3 전체 0.925 = 손만 12개 평균 ✅
C3	Table 1	CHORD vs SPIDER: SP-SR t = 0.58(유의하지 않음), AUC t = 4.2, MP-SR t = 3.2, JPE t = −8.5, RPE t = −1.9(유의하지 않음)
C4	Table 2	Position Only 대비 6개 지표 모두 $p < 0.03$. SP-SR 이득의 41%가 wrench로 바꾼 몫
C5	실물	23/25, Wilson [0.75, 0.98]. 상자 6/7 vs 7/7, Fisher p = 1
C6	Fig. 6	$r^2 = 0.637 \approx$ "약 2/3" ✅

추가 검증 (2026-11-11)

스크립트: `analysis/extra_lid_cem.py` → `analysis/extra_lid_cem_results.json` (numpy 단일 스레드, 약 60초).

추가 검증 1: 보상을 최대화하는 접촉과 뚜껑 열림

목적. §2.2의 보상 곡선을 최적화 결과로 확인한다. 위치 보상, 코드 보상, Eq. 3을 각각 CEM으로 최대화했을 때 어떤 접촉을 고르는지, 그 접촉이 실제로 뚜껑을 여는 방향의 힘을 낼 수 있는지 본다.

설정(가정).

- wrench_toy와 같은 상자(반치수 10 × 7.5 × 5 cm), 기저 512개, μ = 0.1, tol = 0.1, var = 0.1.
- 사람 접촉: 앞면 윗모서리 아래 (0.10, 0, 0.04) m, 법선 +z(엄지가 위로 밂).

- 경첩: 뒤쪽 윗모서리 (−0.10, 0, 0.05) m, 축 y.
- 로봇 접촉 탐색: 여섯 면 전체의 표면점과, 면 안쪽 법선에서 최대 90° 기울인 힘 방향(힘은 물체 안쪽).
- 위치 보상 $\exp(-\|\Delta p\|^2/(2\ \mathrm{cm})^2)$. CEM은 표본 1500개, 25회 반복, 엘리트 10%, 시드 8개.
- 열림 판정: 접촉이 뚜껑 영역($z \geq 0$, 가정)에 있고, 마찰 원뿔($\mu = 0.1$, 8모서리; 0.5로도 확인) 안 힘의 경첩 둘레 여는 방향 토크 최댓값이 0보다 크면 열림. 정적 토크의 부호만 본다. 뚜껑 무게, 경첩 마찰, 상자 고정은 무시한다.
- 탐색 공간을 아랫면을 뺀 뚜껑 영역(옆면은 $z \in [0, 5\ \mathrm{cm}]$)으로 제한한 변형도 봤다.

결과.

보상	탐색 공간	열림 (μ = 0.1 / 0.5)	선택한 접촉과 사람 접촉의 거리 (중앙값)	법선 차이 (중앙값)	보상 (중앙값)
위치	전체	5/8 / 7/8	0.0 cm	81°	1.000
위치	뚜껑 영역	4/8 / 4/8	0.0 cm	111°	1.000
코드	전체	0/8 / 0/8	9.0 cm (아랫면)	0.5°	0.999
코드	뚜껑 영역	8/8 / 8/8	0.14 cm	0.1°	1.000
Eq. 3	전체	0/8 / 0/8	9.0 cm (아랫면)	0.5°	0.862
Eq. 3	뚜껑 영역	8/8 / 8/8	0.18 cm	0.0°	1.000

사람 접촉의 열림 토크는 0.20 N·m/N이다.

결론.

- 코드 보상과 Eq. 3을 전체 표면에서 최대화하면 사람과 같은 법선(+z)을 가진 아랫면 접촉을 고른다. 이 접촉은 뚜껑을 열지 못한다(0/8). 두 접촉의 상자 중심 기준 wrench가 거의 같기 때문이다. 토크를 경계 구 반지름으로 나누므로 접촉 높이 차이가 지지 함수에 거의 반영되지 않는다. 즉 코드 보상은 "어디를"을 거의 보지 않는다. §2.3의 "위치에 둔감하다"는 말을 이 경우로 넓히면, 뚜껑과 바닥을 구분하지 못한다는 뜻이 된다.
- 뚜껑 영역으로 탐색을 제한하면 두 보상 모두 사람 접촉을 거의 정확히 찾고 8/8이 연다. 보상은 방향을 맞추지만, 어디에서 찾을지는 다른 항(추적 보상, 탐색 범위)이 정해야 한다.
- 위치 보상은 위치는 정확히 찾지만 법선을 정하지 못해, 열림이 4–7/8로 흩어진다. §2.2와 맞는다.
- **한계.** 정적 토크 부호 검사다. 뚜껑 영역 $z \geq 0$은 가정이다. 정책 학습(RL)을 하지 않았으므로, 이 탐색 편향이 실제 학습된 정책에서 나타나는지는 확인하지 않았다.

추가 검증 2: 보상별로 학습한 뚜껑 열기 정책 (MuJoCo, 2링크 손가락)

목적. §2.2와 추가 검증 1은 보상을 최적화한 접촉이 뚜껑을 여는지를 정적으로 봤습니다. 이 절에서는 같은 기하에서 정책을 실제로 탐색해, 위치 보상과 wrench(열림 토크) 보상으로 학습한 정책이 뚜껑을 여는 성공률이 달라지는지 봅니다.

설정(가정).

- 상자는 고정된 밑판(반치수 9 × 7.5 × 2.5 cm) 위에 힌지 뚜껑(길이 20 cm, 판 질량 50 g)이 놓인 형태입니다. 뚜껑 앞 턱(lip, 두께 1 cm, 높이 5 cm, 질량 10 g)이 있고, 힌지는 뒤쪽 윗모서리입니다. 뚜껑은 중력만으로 닫혀 있고, 힌지에는 스프링이 없으며 감쇠 0.02만 줍니다.
- 손가락: 평면 2링크(링크 7 cm, 질량 20 g, 반지름 6 mm), 밑동 (0.20, −0.02) m, 위치 서보 kp 50. 끝 구 반지름 8 mm. 손가락은 (0.15, 0) m에서 시작합니다.
- 정책: 손끝 경로를 직선 두 구간으로 정합니다. 시작 → 경유점 g(0.5 s) → 경유점 p(0.5 s). 경유점 4개 좌표(x_g, z_g, x_p, z_p)가 정책 매개변수이고, 관절 목표는 팔꿈치가 위인 역기구학으로 정합니다.

- 탐색: CEM, 표본 40, 엘리트 8, 20회 반복, 초기 표준편차 0.03 m, 경계 $x \in [0.09, 0.17]$, $z \in [-0.03, 0.08]$. 시드 3개.
- 보상 두 가지. 위치 보상: $\exp(-d^2/(2\sigma^2))$, d = 경유점 p와 뚜껑 턱 앞면의 접촉점 (0.108, 0.025) m 사이 거리, σ = 1 cm. wrench 보상: 두 번째 구간(밀기)의 평균 열림 토크(N·m, 힌지 기준, 접촉력으로 계산). 이 보상은 CHORD의 지지 함수(Eq. 3)가 아니고, 접촉에서 열림 방향 토크를 직접 잰 대리 값입니다.
- 성공: 1.0 s 뒤 뚜껑 열림 각 > 0.3 rad.
- `chord/analysis/extra_lid_policy.py` → `chord/analysis/extra_lid_policy_results.json`. CPU 1스레드, 약 36 s. 패키지: mujoco 4.0.0, numpy 1.26.4, Python 3.12.7.

결과. 뚜껑이 닫힌 상태의 기준각은 0 rad입니다(움직이지 않는 손가락).

보상	시드 0	시드 1	시드 2	성공률	끝 손끝과 턱 거리
위치	0.00 rad	0.00 rad	−0.00 rad	0/3	0.5-2 mm
wrench	0.40 rad (성공)	0.17 rad	0.35 rad (성공)	2/3	4-6 cm

- 위치 보상의 최적 정책은 턱 앞면에 손끝을 맞춥니다(보상 0.998-0.9996). 이 접촉은 뚜껑을 열지 못합니다. 뚜껑을 열기 위한 위쪽 힘이 없기 때문입니다.
- wrench 보상의 최적 정책은 턱 아래쪽에서 위로 밀어 올리는 경로를 찾습니다. 시드 0과 2는 뚜껑을 0.35-0.40 rad까지 엽니다. 시드 1은 0.17 rad에서 멈춥니다.

결론.

- 같은 기하와 같은 탐색 예산에서 위치 보상으로 학습한 정책은 0/3, wrench 보상은 2/3 성공입니다. §2.2와 추가 검증 1의 정적 결론(위치 보상은 접촉의 법선·방향을 보지 못함)이 정책 탐색 수준에서도 나타났습니다.
- 다만 이 결과는 설계한 대리 보상(열림 토크)이 최적화 대상이기 때문에 나온 것입니다. CHORD의 지지 보상 자체를 학습에 쓴 것은 아닙니다. 두 보상의 차이가 CHORD 논문의 지지 보상 효과와 같다고 말하려면 그 보상을 직접 넣어야 합니다.
- 시드 1의 실패는 CEM 탐색이 지역 최적에 머문 것으로 보입니다(보상 0.68). 시드 수가 3개이므로 2/3과 0/3의 차이를 통계적으로 강하게 말할 수는 없습니다.
- **한계.** 정책은 경유점 4개뿐이고, 학습은 PPO가 아니라 CEM입니다. 뚜껑 기하, 턱 높이, 손가락 위치는 한 가지뿐입니다. 힌지 마찰, 뚜껑 무게 중심, 상자 고정 오차는 넣지 않았습니다. 성공 기준(0.3 rad)과 1.0 s 시간 제한도 설정입니다. 공식 CHORD 학습과 직접 비교하지 않았습니다.

04. 코드 검증: 논문 ↔ 구현 대조, 수치 재계산, 구현 가능성

- **대상:** `nvidia-isaac/video_to_data` @ `9cad120da522d4086b47052207d7a79ae06a6893` (2026-11-02)의 `robotic_grounding/`. 미디어(mp4 등)를 뺀 sparse clone이고 `repo/`에 있습니다(git 제외). 두 학습 경로를 모두 읽었습니다.
  - **PPO 경로:** Isaac Lab + rsl_rl. `source/robotic_grounding/robotic_grounding/tasks/v2d/` (이하 `v2d/`), `scripts/rsl_rl/train.py --task Sharpa-V2D-v0`
  - **FlashCHORD 경로:** Newton + Warp + JAX, PPO와 FlashSAC. `flash_chord/src/flash_chord/` (이하 `fc/`)
- **범위:** GPU 실행 없이 코드 읽기, 설정 대조, CPU 재계산만 했습니다. 공개 저장소에는 1,831개 결과의 rollout·학습 로그·체크포인트가 없어서 Fig. 5나 Table 1을 결과 파일로 재계산할 수는 없습니다.

1. 모듈 매핑

논문	구현 위치	일치 여부
마찰원뿔 다면체, 단위 모서리 d개 (Eq. 1)	`v2d/mdp/utils.py: compute_friction_cone_edges` (n + μ(cos θ t_1 + sin θ t_2) 정규화, 법선 추가)	✅ 모서리 8개 + 법선 1개 = 9개, μ = 0.1 (D4)
primitive wrench [f, p × f] (Eq. 1)	`utils.py: compute_wrench_space`	✅ 토크를 경계 구 반지름 r_c로 나눔. 논문 식에는 r_c 없음
미리 뽑은 단위 방향 b개	`utils.py: sample_wrench_space_basis_scaled(512, rc=1.0)` (`hand_object_commands.py:751–752`), `fc/configs/objective/tracking.yaml: wrench_basis_seed 0`	✅ 6차원 가우시안 정규화 = 단위 구 균일. body마다 다른 기저
지지 함수 σ = max_col($\mathcal{B}^T\mathcal{W}$) (Eq. 2)	`utils.py: compute_wrench_space_support_function`	✅ 0 아래는 잘라냄
사람 쪽 σ_h	`hand_object_commands.py: _precompute_contact_wrench_support_values` (리타겟 데이터의 접촉 위치·링크 법선, 프레임별 미리 계산)	✅
로봇 쪽 σ_r	`utils_jit.py: wrench_preprocess_jit` → `wrench_support_one_body_jit` (시뮬레이터 접촉력 방향을 법선으로)	✅ 힘 크기는 정규화로 사라짐 (D7)
접촉 wrench-space 보상 (Eq. 3), β, v	`utils_jit.py: contact_wrench_support_reward_jit`, `tolerance 0.1`, `var 0.1`, 가중치 10 (`v2d_hand_env_cfg.py:251–258`)	⚠️ 식이 다름 (D1)
r_unintend, r_miss	`unintended_contact_penalty_jit` (가중치 −10), `missed_contact_penalty_jit` (−1)	✅ unintended는 body 단위 (D10)
r_fc (Sec. 3.2)	`v2d/mdp/rewards.py: force_closure_reward(min_support=0.01)`, 가중치 0	⚠️ 손별 계산, 기본 꺼짐 (D8)
r_task (물체 자세 exp)	`object_keypoints_tracking_exp` (var 0.1)	✅ SE(3) ⊖ 대신 keypoint 거리
r_relative (두 물체 상대 자세, App. B)	`rewards.py: relative_object_pose_reward` 등 정의만 있음	❌ 기본 보상 설정 어디에도 등록되지 않음 (D11)
r_imit	`hand_keypoints_tracking_exp` (0.25), `hand_joint_pos_tracking_exp` (0.25)	✅ keypoint에 포화 임계값 있음

논문	구현 위치	일치 여부
VOC 커리큘럼	`v2d/mdp/curriculum.py: FixedTimestepCurriculum`, `fc/lifecycle/curriculum.py: FixedCurriculum`	✅ 같은 `bisect_right` 의미, FlashCHORD는 10배 빠름 (C7)
리셋: 임의 프레임, 손가락 [0, 0.7] 보간, VOC 20스텝	`v2d_hand_env_cfg.py:63–67` (`reset_finger_openness 0.7`, `virtual_object_control_decay_steps 20`)	✅
행동 배율 0.05 m, 0.15 rad, 0.15 rad, EMA 0.3	`v2d/mdp/actions/actions_cfg.py:39–57`	✅
제어 20 Hz	`decimation = 5`, `sim.dt = 0.01`	✅
FlashSAC, 환경 2,048개, L40S 약 2시간	`fc/configs/experiment/sharpa_flash_sac.yaml` (`world_count 4096`, 2.5억 전이)	⚠️ 환경 수 다름 (D6), 시간은 확인 불가
성공: 15 cm / 40° 이탈 없이 끝까지, 완료율 > 0.7	`fc/evaluation/metrics.py: chord_success`, `evaluation/harness.py: evaluation_thresholds`	⚠️ 임계값 다름, 0.7 규칙 없음 (D3)
전신 확장	`v2d_whole_body/`, `fc/configs/experiment/g1_recon_body_*.yaml`	읽기만 함(세부 대조 안 함)
GR00T 후속 학습	`groot_finetune/` (전문가 rollout → LeRobot → GR00T)	논문에 없는 부분

2. 불일치와 논문에 적히지 않은 구현 요소

#	내용	영향
D1	**보상 식이 Eq. 3과 다릅니다.** 코드는 방향별 loss_b를 따로 exp한 뒤, 사람·로봇 지지값이 모두 10^{-3} 초과인 칸만 더해 사람 활성 방향 수로 나눕니다(`utils_jit.py:472–526`). 그 뒤 body 평균, 손 평균입니다. Eq. 3은 512방향 위반 합의 exp입니다.	toy에서 Eq. 3은 법선 10°에서 0.004, 코드는 0.95입니다(`03` §2). 논문 식대로 재구현하면 보상이 거의 0이라 학습이 안 될 수 있습니다.
D2	**공개 순서와 논문 주장의 시점이 어긋납니다.** 첫 학습 코드 v0.2.0(`cbee067`, 2026-08-14)은 PPO 경로만 있습니다. FlashCHORD는 v0.4.0(`33129dd`, 2026-09-23)에 처음 들어왔습니다. 논문 v2(2026-08-14)는 FlashSAC으로 2시간이라고 적었습니다. 82.12%·90.77%와 그림 파일은 v1과 같습니다. FlashCHORD의 `reference_curriculum()`은 "W&B 실행 `sizirwre`에서 복사"이고 값이 PPO 설정과 같습니다.	FDR이 PPO 경로로 CHORD를 돌린 것은 당시 공개 상태와 맞습니다(`02` §3). 1,831개 결과가 어느 경로에서 나왔는지는 밝혀져 있지 않습니다.
D3	**종료·성공 임계값이 세 가지입니다.** 논문: 물체 15 cm / 40°. PPO 경로: 손목 0.2 m, 물체 0.2 m / 0.7 rad(40.1°) (`v2d_hand_env_cfg.py:303–318`). FlashCHORD: 손목 0.15 m, 물체 0.10 m / 0.50 rad(28.6°) (`fc/configs/termination/tracking.yaml`). FlashCHORD 평가는 체크포인트 설정의 종료 임계값을 그대로 성공 판정에 씁니다 (`harness.py: evaluation_thresholds`). 완료율 > 0.7을 과제 성공으로 세는 코드는 찾지 못했습니다.	논문의 40°는 PPO의 0.7 rad, 15 cm는 FlashCHORD의 손목 값과 숫자가 맞습니다. GNR이 "CHORD 관례"로 쓴 HOT3D 150 mm는 논문 숫자입니다. 코드로 같은 판정을 재현하려면 임계값을 직접 맞춰야 합니다.
D4	**μ = 0.1, 모서리 8 + 법선 1, 방향 512개, 토크/r_c는 논문에 없습니다** (`commands_cfg.py:107–113`, `fc/configs/objective/tracking.yaml:62–65`). μ = 0.1은 원뿔 반각 5.7°입니다.	재구현에 꼭 필요한 값입니다. μ에 대한 보상 민감도는 작습니다(`03` §2.5).

#	내용	영향
D5	**추적 가중치 일정이 두 경로에서 다릅니다.** PPO: 물체 keypoint 가중치 0 → 0.1 → 0.25 → … → 1 → 20(마지막 단계), 손 keypoint·관절 0.25. FlashCHORD: 물체 keypoint 20을 처음부터. 논문에는 가중치가 없습니다.	PPO 경로는 학습 초반에 물체 추적 보상이 없고 wrench 보상(가중치 10)만으로 접촉을 배웁니다. 논문의 "VOC를 줄이기 전에 wrench 보상이 먼저 수렴해야 한다"(Sec. 4.1)는 이 일정과 맞습니다.
D6	**환경 수:** 논문 App. B는 2,048개, Table 2 캡션은 "시드당 4,096개", 코드는 두 경로 모두 4,096개 (`v2d_hand_env_cfg.py:386`, `sharpa_flash_sac.yaml:26`).	2시간 주장이 어느 설정인지 불분명합니다.
D7	**힘 크기를 보지 않습니다.** 로봇 쪽 법선은 접촉력 방향이고(`wrench_preprocess_jit`), `compute_wrench_space`가 정규화합니다. 모서리도 단위 길이입니다.	0.1-10배 힘에서 지지값이 같습니다 (`03` §3). DexForce와의 핵심 차이입니다(`02` §4).
D8	**r_fc는 손별이고 기본 꺼짐입니다.** body를 amax로 합친 뒤 손 하나의 지지값으로 비율을 계산하고, 참조 접촉 레이블로만 켭니다. 가중치 0이며 "잡음 있는 레퍼런스에서는 켜고 접촉 항 셋을 0으로 하라"는 주석이 있습니다(`v2d_hand_env_cfg.py:276–285`, `rewards.py:627`).	양손으로만 force closure가 되는 물체는 손별 r_fc가 1에 닿지 못합니다. 512개 표본이라 force closure의 필요조건일 뿐입니다(`03` §4).
D9	**시연을 절반 속도로 재생합니다.** `motion_speed 0.5`(`commands_cfg.py:64`, `v2d_hand_env_cfg.py:63`, `fc/configs/task/default.yaml:4`, `sharpa_flash_sac.yaml:20`). 전신 G1 설정은 1.0입니다.	논문에 없습니다. Fig. 7의 "40-48초"가 원본 시간인지 알 수 없고, 실물 재생 속도에도 영향을 줍니다. Do as I Do도 실물을 약 0.5배속으로 재생했습니다(`do_as_i_do/01` §2).
D10	**같은 body 위의 추가 접촉은 약하게만 벌합니다.** 보상은 사람 지지값이 0인 방향을 세지 않고, r_unintend는 사람 접촉이 없는 body에만 걸립니다.	toy에서 옆면 추가 접촉은 보상 1.00 → 0.87로만 떨어집니다(`03` §2.4).
D11	**r_relative가 꺼져 있습니다.** 함수는 있지만(`rewards.py:487–560`) 기본 RewardsCfg와 FlashCHORD 목적 함수 어디에도 등록되지 않았습니다. 커리큘럼 인자 기본값도 0입니다 (`curriculum.py:132–133`).	논문이 다물체 과제(삽입·붓기)의 핵심으로 설명한 항입니다. 공개 설정으로는 Table 1의 다물체 19개 결과가 이 항 없이 나온 것인지 알 수 없습니다.
D12	**시드 수:** Table 1 캡션은 3개, App. G는 DexMachina·SPIDER 시드 5개.	표 오차 막대의 의미가 기준선마다 다를 수 있습니다.
D13	**SP-SR의 위치 오차 정의가 구현마다 다릅니다(2026-11-12 추가).** CHORD 공개 평가 코드 `fc/evaluation/metrics.py: spider_success`는 시간 평균한 원 위치 오차를 10 cm·0.5 rad와 비교하고, docstring에 "원 구현의 시간 평균 빼기는 생략한다"고 적었습니다. 중심화한 오차(`spider_centered_position_error`)는 진단 값(`spider_position_error_centered_m`)으로만 기록합니다. SPIDER 원 스크립트는 양손 모드에서 궤적 평균을 빼고(`flashdexretarget/04` D3), V2D 챌린지 키트의 트랙 2 `metric_code/track_2/SP-SR.py`도 같은 원 오차를 씁니다(`_ALIGN_INITIAL_POSE = False`). 첫 프레임 강체 정렬(`_warp_to_reference_start`)은 트랙 3 파일에서만 켜집니다(`v2d_corl_tutorial/04` D2). 다음 커밋 `5226e34`의 `spider_success`도 같습니다.	Table 1이 어느 정의로 계산됐는지는 논문에 없습니다. 처음 분석에서 SP-SR을 "궤적 평균으로 중심화하는 관대한 지표"라고 한 것은 SPIDER 원 스크립트 기준이었고, CHORD 공개 코드와는 맞지 않아 `01` §3.3, `02` 표와 §3.4, 블로그를 고쳤습니다. SPIDER 대비 동률(t = 0.58)이라는 통계 결론은 그대로입니다.

3. 주장별 재계산

`table_check.py`, `wrench_toy.py` 결과입니다. 자세한 해설은 `03`에 있습니다.

#	주장 (출처)	재계산	판정
C1	1,831개 평균 82.12% (Abstract, Fig. 5)	n 합 1,831. 막대 n 가중 평균 0.781, 단순 평균 0.801. 82.12% × 1,831 = 1,503.6(정수 아님), 4,511/5,493 = 82.12%(시드 3개면 맞음). 평가의 70%가 HOT3D	⚠️ 집계 방식 미기재
C2	전신 90.77% (Abstract, Fig. 5 오른쪽, Table 3)	17개 가중 평균 0.9077, Table 3 전체 = 손만 12개 평균 0.925	✅
C3	모든 지표에서 기준선보다 낫다 (Table 1)	SPIDER 대비 SP-SR t = 0.58(p = 0.60), RPE t = −1.9(p = 0.17)는 유의하지 않음. AUC·MP-SR·JPE·MPPE는 p < 0.05	⚠️ SPIDER 지표에서는 동률
C4	접촉 지도가 풍부할수록 좋다 (Table 2)	Position Only 대비 6개 지표 모두 p < 0.03, SP-SR 이득의 41%가 wrench 몫. CHORD 행은 Table 1과 같은 실행	✅
C5	실물 전이 (Sec. 4.6)	23/25, Wilson [0.75, 0.98]. 상자 open 6/7 vs closed 7/7, Fisher p = 1	✅ (표본 작음)
C6	CWS 보상이 성공과 상관 (Fig. 6)	$r^2 = 0.637 \approx 2/3$	✅ (인과 아님)
C7	과제당 약 2시간, 기준선 4-20시간 (App. B)	FlashCHORD 2.5억 전이 = 2시간이면 34.7k/s. PPO 경로는 19.7억 전이. FDR 측정 PPO 모션당 57-66 GPU-h	⚠️ PPO 경로에는 해당 안 됨
C8	15 cm / 40° 성공 판정 (Sec. 4.1)	코드 PPO 0.2 m / 0.7 rad, FlashCHORD 0.10 m / 0.50 rad	❌ 코드와 다름 (D3)
C9	지지 함수 비교가 위치 보상보다 접촉을 잘 구분 (Sec. 3.1 그림)	같은 위치, 법선 90° 차이: 위치 보상 1.00, 코드 보상 0.33	✅
C10	r_fc 최대화 = force closure (Sec. 3.2)	두 점 집기(force closure 아님)에서 r_fc = 1, 표본 ε = 0.053	❌ 512개 표본에서는 필요조건일 뿐
C11	Ferrari-Canny ε = min 지지값 (App. D)	512개 방향의 덮는 각도 중앙값 22°, 최대 38°. 표본 최솟값은 진짜 ε의 상한	⚠️ Fig. 3 ε 분포는 상한
C12	wrench는 힘·토크로 비교 (Abstract)	힘 크기 0.1-10배에서 지지값 동일	⚠️ 방향만 비교
C13	v1 ↔ v2	그림 11개 바이트 동일, 초록 수치 동일, Table 1·2와 학습 설정 단락은 v2에서 새로 씀 (`01` §5)	기록
C14	프로젝트 페이지 `chord.pdf` = arXiv v2	단어 12,075개 중 날짜 도장만 다름	✅

4. 구현할 수 있는지

4.1 공식 재현: 이 맥에서는 불가, NVIDIA GPU가 있으면 가능

- **이 맥(Apple M4 Pro, 메모리 약 24 GiB, MPS, CUDA 없음):** 두 경로 모두 실행할 수 없습니다. Isaac Lab 경로는 Docker + NVIDIA Container Toolkit + 드라이버 580(CUDA 13 권장)이 전제입니다(`robotic_grounding/README.md`). FlashCHORD는 Linux, 드라이버 580 이상, 전체 학습에 **48 GiB GPU와 여유 46,000 MiB 이상**을 요구합니다 (`flash_chord/README.md`). Newton·Warp에 CPU 장치가 있지만 FlashCHORD는 Docker·NVIDIA 전제로 짜여 있어 맥 CPU 실행은 공식 경로가 아닙니다(시도하지 않음).
- **NVIDIA GPU가 있으면:** 코드(Apache-2.0)와 과제 라이브러리(Hugging Face, CC BY 4.0)가 공개되어 있습니다. `tissue_box_simple` 시퀀스 하나는 리타겟까지 끝난 상태로 들어 있어 바로 학습해 볼 수 있습니다. 원천 데이터셋과 MANO는 각자 라이선스 동의 후 직접 받아야 합니다.
- **비용 견적:** FlashCHORD 논문 기준 과제당 약 2 L40S-h이면 Table 1(60개 × 시드 3) = 360 GPU-h, 1,831개 한 번 = 약 3,700 GPU-h입니다. PPO 경로는 FDR 측정 기준 모션당 57-66 RTX 3090 GPU-h입니다. 24 GiB 소비자 GPU는

FlashCHORD 전체 학습 요구(48 GiB)를 채우지 못하고, PPO 경로는 가능합니다(FDR이 RTX 3090으로 돌림).

- **재현 시 맞춰야 할 것:** 종료·성공 임계값(D3), 완료율 0.7 집계(C1), r_relative 활성화(D11), 환경 수(D6), 시드 수(D12). 이것들이 논문과 코드에서 달라서 "공식 코드를 그대로 돌리면 논문 수치가 나온다"고 기대하기 어렵습니다.

4.2 아이디어 재구현: 쉽고, 이미 핵심은 numpy로 옮겼음

- 핵심 보상(마찰원뿔 → wrench 행렬 → 지지 함수 → 보상·벌칙)은 numpy로 약 80줄입니다(`analysis/wrench_toy.py`). torch로 옮기면 어느 시뮬레이터의 접촉점·접촉력 출력에도 붙일 수 있습니다. MuJoCo의 `mj_contactForce`나 MJX가 주는 접촉점·힘 방향이면 충분합니다.
- 재구현 시 권장: Eq. 3이 아니라 코드식(방향별 exp 평균)을 쓰고, μ·기저 개수·r_c 정규화를 명시하고, force closure 판정은 표본 최솟값 대신 LP나 더 많은 방향으로 확인합니다.
- 정책 학습까지 포함한 축소판(한 손 + 상자 뚜껑, MuJoCo CPU)은 이 맥에서도 가능합니다. mujoco 설치가 필요하고, 정책 학습은 numpy CEM이나 torch(MPS)로 할 수 있습니다.

4.3 이 맥에서 이미 한 것과 할 수 있는 것

작업	상태
표·그림 재계산(C1-C6, C13-C14), v1 ↔ v2 단어·그림 대조	완료
공식 보상 함수 numpy 이식, 상자 뚜껑·위치 이동·힘 크기·μ·force closure toy(W1-W9)	완료
학습 예산 산술과 FDR 수치 정합(C7, `02` §3)	완료
공개 과제 라이브러리 일부(수 MB)를 받아 접촉 지지값 분포·표본 ε 상한 효과 측정	가능(다운로드 승인 필요)
MuJoCo 한 손 축소 실험	가능(mujoco 설치 승인 필요)
공식 학습 재현	불가(NVIDIA GPU 필요)

5. 재현 정보 체크리스트

항목	논문	코드
보상 가중치	없음	있음(D5)
β, v	기호만	0.1, 0.1
μ, 모서리 수, 방향 수	없음	0.1, 8 + 1, 512
환경 수	2,048 / 4,096 혼재	4,096
총 학습량	"약 2시간"	PPO 19.7억, FlashCHORD 2.5억 전이
종료 임계값	15 cm / 40°	0.2 m / 0.7 rad 또는 0.10 m / 0.50 rad
과제 목록(1,831개, 60개)	없음	없음(데이터셋에 과제 전체는 있음)
학습 로그·체크포인트	없음	없음
시드	3(Table 1) / 5(App. G)	설정 인자

02. 선행연구와 관련 연구

출처 표기: [n]은 CHORD v2 참고문헌 번호입니다. "내 판단"이라고 쓴 곳은 논문에 없는 해석입니다.

1. 계보도

```
사람 손–물체 시연 → 로봇손 정책 (물리 시뮬레이션에서 추적)
  ├─ 운동학 리타겟 + 잔차 RL: ManipTrans [2] (CVPR 2025, 사전학습 imitator + 잔차)
  ├─ 물체 보조 커리큘럼 RL: DexMachina [1] (ICML 2026, VOC·위치 기반 접촉 보상·ADD-AUC)
  ├─ sampling 최적화: SPIDER [3] (MPPI식 annealing, SP-SR 지표) → Do as I Do (단안 영상 + SPIDER,
이 저장소 do_as_i_do/)
  ├─ 사람 시연 1개 + sim-to-real RL: Human2Sim2Robot [43]
  └─ 접촉 wrench로 지도: CHORD (이 논문, 2026-06)
        └─ 같은 데이터·전처리로 다중 레퍼런스 off-policy: FlashDexRetarget (arXiv:2610.01849, 이 저장소
flashdexretarget/)

접촉 wrench 공간·파지 품질
  Ferrari–Canny [32] → wrench 기반 파지 보상 RL [33–36] (정적 안정성 최적화)
  → CHORD: 정적 안정성이 아니라 "사람 접촉과 같은 물체 운동을 만드는 능력"을 비교 (Sec. 2)

전신 확장 (NVIDIA 같은 그룹)
  SONIC [20] (전신 추적 제어기) + MotionBricks [37] (모션 생성) + CARI4D [38] (3인칭 손–물체 복원)
```

2. 구성 요소별 출처

구성 요소	출처	이 논문에서의 역할
VOC(가상 물체 제어) 커리큘럼, 위치 기반 접촉 보상, ADD-AUC	DexMachina [1]	학습 보조와 "Position Only" ablation, AUC 지표
잔차 행동 공간, MP-SR 지표	ManipTrans [2]	리타겟 동작을 사전으로 쓰는 잔차 정책
SP-SR 지표	SPIDER [3], `spider/postprocess/get_success_rate.py`	시간 평균 위치 오차 ≤ 10 cm, 자세 ≤ 0.5 rad. SPIDER 원 스크립트는 양손 모드에서 궤적 평균을 빼지만 (`flashdexretarget/04` D3), CHORD 공개 평가 코드와 V2D 챌린지 키트는 빼지 않습니다(`04` D13, 2026-11-12 수정)
시뮬레이터	Isaac Lab [42] (PPO 경로), Newton + Warp + JAX (FlashCHORD 경로)	코드에 두 경로가 함께 있음(`04` §1)
PPO 구현	rsl_rl (`scripts/rsl_rl/train.py`)	공개 첫 학습 경로
FlashSAC	FlashSAC (Holiday Robotics·KAIST)	v2 App. B가 말하는 학습기. FlashCHORD에 JAX로 들어 있음
데이터	ARCTIC, TACO, HOT3D, OakInk2, DexYCB, GRAB, H2O [13-16, 39-41]	4,739개 과제의 원천
영상 복원	MoGe, Grounding DINO, SAM2, SAM3D, FoundationPose, ViPE, Dyn-HaMR, Gaussian splat, DROID-SLAM [45-53]	App. C 붓 과제 한 건

구성 요소	출처	이 논문에서의 역할
전신	SONIC [20], MotionBricks [37], CARI4D [38]	잔차를 얹을 전신 제어기와 손 → 전신 inpainting
파지 품질	Ferrari-Canny ε [32]	벤치마크 통계(Fig. 3), r_fc의 근거

3. FlashDexRetarget과의 정합: CHORD 자신의 논문은 FDR이 돌린 CHORD와 맞는가

`flashdexretarget/`에서 CHORD는 주 기준선이었습니다. XHand 50개 모션에서 SR_SPIDER 0.46, 2,847 GPU-h(RTX 3090), Sharpa에서 0.50, 3,314 GPU-h였고, FDR은 이를 근거로 "약 100배 적은 계산량, 44%p 높은 성공률"을 주장했습니다(FDR Table I, `flashdexretarget/01` §3.2). CHORD 논문과 코드로 이 비교를 다시 보면 다음과 같습니다.

3.1 FDR이 돌린 것은 CHORD의 PPO 공개 경로입니다

- FDR Fig. 1 캡션과 Sec. IV-A는 "CHORD 결과는 공개 구현으로 얻었고, PPO로 학습한다"고 적었습니다. FDR v1은 2026-10-01에 올라왔고, video_to_data의 첫 학습 코드 공개(v0.2.0)는 2026-08-14, FlashSAC 경로(FlashCHORD, v0.4.0)는 2026-09-23입니다. FDR의 기준선 실험은 PPO 경로만 있던 시기에 돌았을 가능성이 큽니다(실험 날짜는 FDR에 없음).
- PPO 공개 설정은 환경 4,096개 × 24스텝 × 최대 20,000 iteration = **약 19.7억 전이**이고, VOC는 iteration 12,500(약 12.3억 전이)부터 0입니다(`04` C7, `curriculum.py`의 `bisect_right` 의미로 계산).
- FDR이 잰 모션당 비용은 2,847/50 = 56.9 GPU-h(XHand), 3,314/50 = 66.3 GPU-h(Sharpa)입니다. 20,000 iteration을 다 돌렸다고 가정하면 RTX 3090에서 초당 약 9.6k·8.2k 전이입니다. Isaac Lab에서 양손 접촉 센서와 wrench 계산을 매 스텝 하는 설정으로는 그럴듯한 값입니다. 다만 이 처리량은 GPU-h에서 거꾸로 계산한 값이라 독립적인 확인은 아닙니다.

3.2 CHORD v2가 주장하는 레시피는 FlashSAC이고, 10배 짧습니다

- App. B: FlashSAC, 환경 2,048개, L40S 한 장에서 과제당 약 2시간. FlashCHORD 설정은 2.5억 전이이고(`sharpa_flash_sac.yaml`), VOC 임계값 9개가 PPO 일정을 정확히 1/10로 줄인 값입니다. 설정 파일 주석도 "검증된 10배 VOC 일정을 유지한다"입니다. 2시간에 2.5억이면 초당 약 34.7k 전이입니다.
- FDR이 잰 모션당 57-66 GPU-h는 논문의 2시간보다 **28-33배** 큽니다. 전이 수 비율(19.7억 / 2.5억 = 7.9배)과 처리량 차이(3090 + PhysX vs L40S + Newton/Warp, 약 3.6-4.2배)를 곱하면 이 간극이 설명됩니다.
- 가정으로 CHORD-FlashSAC을 50개 모션에 돌리면 약 100 L40S GPU-h입니다. FDR의 29(XHand)·44(Sharpa) RTX 3090 GPU-h와 비교하면 **2.3-3.4배**입니다. 하드웨어가 달라 정확한 비율은 아니지만, FDR의 "약 100배"는 주로 PPO 대 off-policy, 그리고 PPO 경로의 긴 기본 일정에서 나온 것입니다. 같은 FlashSAC을 쓰는 CHORD와 비교했다면 격차는 한 자릿수였을 것입니다. FDR 저자는 FlashSAC 저자와 같은 그룹이라, FlashCHORD가 FlashSAC을 쓴다는 점이 이 해석과 맞물립니다.

3.3 그런데 CHORD의 대규모 수치 자체가 어느 레시피에서 나왔는지는 불분명합니다

- 82.12%(1,831개), 90.77%(전신), Fig. 5-9의 그림 파일은 v1(2026-06-22)과 v2가 **바이트 단위로 같습니다**. FlashSAC 학습 단락은 v2에서 처음 나옵니다. 1,831개 평가가 PPO로 학습되었다면, FDR이 돌린 PPO 경로는 오히려 CHORD 자신의 대규모 결과를 만든 레시피에 가깝습니다. FlashSAC으로 학습되었다면 FDR의 비교는 CHORD의 최신 레시피를 반영하지 못한 것입니다. 논문은 어느 쪽인지 밝히지 않습니다.
- FlashCHORD의 `reference_curriculum()`은 "W&B 실행 `sizirwre`에서 복사"했다고 적혀 있고, 그 값이 PPO 설정과 같습니다. PPO 일정이 먼저 검증되고 FlashSAC으로 옮겨졌다는 정황입니다(내 판단).

3.4 성공률 차이: 과제·로봇·지표가 모두 다릅니다

항목	CHORD 논문 Table 1	FDR Table I의 CHORD
과제	자체 60개(강체 17, 관절 24, 다물체 19)	FDR 50개(HOT3D 25, TACO 10, OakInk2 15)
로봇	Sharpa Wave(CHORD 기본 손)	XHand(CHORD 기본 손이 아님), Sharpa Wave
학습	논문은 FlashSAC이라고 함	PPO 공개 경로
SPIDER 지표	SP-SR **0.926**	SR_SPIDER **0.46**(XHand), **0.50**(Sharpa)
엄격한 지표	MP-SR 0.894 (ManipTrans식)	SR_MT 0.00, SR_MT-Obj 0.02(XHand)·0.22(Sharpa)
판정	완료율 > 0.7 (rollout 여러 개의 비율)	FDR 쪽 판정(첫 성공에서 멈추고 실패 모션은 전체 비용 합산)

- 같은 Sharpa 손, 같은 이름의 SPIDER 지표에서도 0.926 vs 0.50입니다. 과제 묶음, 학습 레시피, 성공 판정(완료율 > 0.7 vs FDR의 rollout 판정)이 모두 달라서 둘 중 하나가 틀렸다고 할 수는 없습니다.
- 위치 오차 정의도 다릅니다(2026-11-12 추가). FDR의 SR_SPIDER는 양손에서 궤적 평균을 빼고(`flashdexretarget/04` D3), CHORD 공개 평가 코드는 빼지 않습니다(`04` D13). 평균을 빼면 오차가 줄어드는 쪽이므로, 이 정의 차이로는 FDR 쪽 값이 더 낮은 것을 설명할 수 없습니다.
- 가장 큰 불일치는 엄격한 지표입니다. CHORD 자신은 MP-SR 0.894를 보고했는데, FDR에서는 SR_MT가 0입니다. FDR의 SR_MT는 손끝 6 cm, 관절 8 cm 조건까지 요구하고(`flashdexretarget/01` §3.1), CHORD의 MP-SR이 이 손 조건을 포함하는지는 논문에 적혀 있지 않습니다. 같은 이름의 지표라도 구현이 다를 수 있습니다.
- FDR 쪽의 불리한 요인: XHand는 CHORD의 기본 손이 아니므로 접촉 링크 대응, 마찰원뿔(μ = 0.1), 보상 가중치를 다시 맞춰야 했을 수 있습니다. FDR이 XHand용으로 튜닝했는지는 알 수 없습니다(`flashdexretarget/01` §3.2 "기준선이 모두 공개 구현 그대로").
- CHORD 쪽의 유리한 요인: Table 1의 60개 과제는 1,831개에서 고른 것이고 선택 기준이 없습니다. 기준선은 모두 CHORD 팀이 Sharpa로 옮겨 다시 구현했습니다.

결론: 두 논문은 모순되지 않지만, 서로의 수치를 옮겨 쓸 수 없습니다. FDR의 "100배"는 "CHORD의 PPO 공개 경로 대비"로 읽어야 합니다. CHORD의 "과제당 2시간"은 FlashCHORD 경로의 주장이고, 그 경로로 1,831개 결과를 만들었다는 증거는 논문에 없습니다. 공정한 비교는 FlashCHORD 경로로 FDR의 50개 모션을 Sharpa에서 돌리는 것입니다.

4. 접촉 보상 세 가지 비교: CHORD wrench, FDR 거리·점군, DexForce 힘 목표

	CHORD (이 논문)	FlashDexRetarget	DexForce
접촉 정보의 출처	사람 MANO 접촉점·법선(시연에서 추출)	사람 손끝-물체 최근접 꼭짓점 거리	손끝 6축 F/T 센서로 잰 실제 힘
무엇을 맞추나	512방향 wrench 지지값(접촉 위치·법선이 만드는 힘·토크 방향 능력)	손끝-물체 거리의 한쪽 상한(Eq. 7) + 물체 점군 worst-k 정합(Eq. 6)	위치 목표 x_f = x_o + k_f f (힘을 위치 오프셋으로 환산)
쓰는 곳	RL 보상(시뮬레이션)	RL 보상·관측(시뮬레이션)	모방학습 행동 목표 + 임피던스 제어(실물)
힘의 크기	**보지 않음.** 접촉력 방향만 쓰고 모서리를 단위 길이로 만듦(`04` D7)	보지 않음(거리만)	**핵심.** 힘 크기가 목표 오프셋을 정함
법선 방향	핵심. 법선이 다르면 보상 하락	보지 않음(SDF 법선은 관측으로만)	센서가 잰 힘 방향으로 간접 반영
위치 오차	둔감(2 cm 이동에도 0.94, `03` §2)	직접 벌함(거리 커널 σ = 1 cm)	관측 위치 그대로
사람 시연 잡음	약함. 그래서 r_fc 대안을 둠	한쪽 거리 상한이라 비교적 관대	시연 자체가 로봇손 kinesthetic이라 잡음 작음

내 판단:

- 세 방법은 같은 축의 강약이 아니라 **다른 축**을 봅니다. CHORD는 "어느 방향으로 밀 수 있는가", FDR은 "어디에 닿아 있는가와 물체가 맞게 움직였는가", DexForce는 "얼마나 세게 미는가"입니다.
- CHORD 보상이 힘 크기를 보지 않는다는 점이 DexForce와의 가장 큰 차이입니다. 시뮬레이션에서는 크기를 물리 엔진과 물체 추적 보상이 정해 주므로 문제가 덜하지만, 실물에서 쥐는 힘이 약해 떨어뜨리는 실패(Sec. 4.7 원격조종의 "fragile grasp")를 wrench 지지값만으로는 막지 못합니다.
- 결합 가능성: CHORD의 지지값 비교에 DexForce식 크기 항(사람 쪽 힘 추정이나 센서 측정치)을 곱하면 "방향 + 크기" 보상이 됩니다. FDR의 점군 정합은 물체 결과를, CHORD의 wrench는 접촉 원인을 맞추므로 둘은 보완 관계입니다. FDR ablation에서 상호작용 보상을 빼면 −7개였는데, 그 자리에 wrench 보상을 넣는 실험은 두 논문 모두 하지 않았습니다.

5. 이 저장소의 다른 논문과의 관계

논문	관계
FlashDexRetarget (`flashdexretarget/`)	§3에서 정리했습니다. FDR은 CHORD의 전처리(TACO·OakInk2·HOT3D)를 그대로 쓰고, CHORD를 주 기준선으로 삼았습니다. CHORD가 "시퀀스 하나에 정책 하나"로 남긴 비용 문제를 다중 레퍼런스 정책 하나로 풀었습니다. 두 방법 모두 FlashSAC을 쓰게 되면서 남는 차이는 보상(wrench vs 거리·점군)과 정책 구조(시퀀스별 vs 공동)입니다.
Do as I Do (`do_as_i_do/`)	같은 Sharpa Wave 손, 같은 "사람 영상 → 로봇 궤적" 문제입니다. Do as I Do는 SPIDER식 sampling MPC에 warmup·교란·전이 보상을 더했고, CHORD는 RL에 wrench 보상과 VOC를 썼습니다. 두 논문 모두 "레퍼런스가 잡음일 때 접촉 사건을 어떻게 유도하나"를 다루는데, Do as I Do는 접촉 유무의 상수 벌칙, CHORD는 r_miss·r_unintend와 r_fc로 답했습니다. CHORD Table 1에서 SPIDER가 관절 오차 41°였다는 결과는 sampling 계열이 사람 손 모양을 보존하지 않는다는 점을 보여 주고, 이는 Do as I Do에도 해당할 수 있습니다(내 판단).
DexForce (`dexforce/`)	§4에서 비교했습니다. 접촉의 "방향"(CHORD)과 "크기"(DexForce)가 서로 빠진 부분입니다. DexForce에서 force-informed 목표 대신 관측 위치를 그대로 행동으로 쓰면 성공률이 거의 0%였던 결과(`dexforce/01` §1)와 CHORD 원격조종 실험의 "쥐는 힘이 약해 실패"(Sec. 4.7)는 같은 현상을 다른 쪽에서 본 것입니다.
Contact-Grounded Policy (`contact_grounded_policy/`)	CGP는 촉각과 실제 상태를 예측해 compliance 제어기 목표로 바꾸고, CHORD는 시뮬레이션에서 접촉 wrench를 맞춥니다. CHORD 실물 배포는 Vicon 상태만 쓰고 촉각이 없습니다(저자 한계 1). CHORD 정책 관측에 이미 "손목 좌표계 접촉 위치·힘 방향"이 있으므로(App. B), 실물에서 이 관측을 촉각 센서로 대신하는 것이 CGP와 이어지는 자연스러운 다음 단계입니다.
GNR (`gnr/`)	GNR은 HOT3D 성공 기준으로 위치 150 mm를 "CHORD 관례"로 썼습니다(`gnr/04` C12). 그 값은 CHORD 논문의 15 cm이고, CHORD 코드는 PPO 경로 0.2 m, FlashCHORD 경로 0.10 m라서 코드 기준으로는 같은 관례가 아닙니다(`04` D3). GNR(샘플링 MPC 궤적으로 학습한 생성 잔차)과 CHORD(시퀀스별 RL)는 같은 HOT3D 데이터에서 "새 시연마다 얼마나 계산하나"의 반대쪽 끝입니다.
Hydra-0 (`hydra0/`)	같은 NVIDIA의 "로봇 데이터 부족" 해법이지만 경로가 다릅니다. Hydra-0은 생성 world model로, CHORD는 물리 시뮬레이션으로 검증된 궤적으로 접근합니다. CHORD 산출물은 Hugging Face에 LeRobot 형식 로봇 데이터로 공개되어 있고, 코드에 GR00T 후속 학습 경로(`groot_finetune/`)가 있습니다. 이 데이터가 Hydra-0 같은 world model의 학습 데이터가 될 수 있습니다(내 판단).

6. 논문이 다루지 않았지만 관련 있는 연구 (내 판단)

연구	관련성	근거
Task wrench space (Li & Sastry 1988), 과제 지향 파지 품질	CHORD의 "사람 wrench 집합과 같은 집합"은 과제 wrench 공간을 시연에서 얻는 방식입니다. 과제 wrench를 포함하는지(⊇)만 보는 고전 지표와 달리 CHORD는 위로도 벌합니다(too_large).	내 사전 지식. 서지 확인 필요
Soft-finger·비틀림 마찰 모델	코드는 점접촉 마찰원뿔만 씁니다. 두 점 집기의 축 둘레 토크(`03` §4)는 soft finger 모델이면 생깁니다. 사람 손가락 패드는 soft finger에 가깝습니다.	내 사전 지식
준균일 방향 집합(Fibonacci·Sobol 구면 점)	512개 가우시안 무작위 방향 대신 준균일 점을 쓰면 같은 개수로 덮는 각도가 줄어듭니다. 6차원에서 512개 무작위 방향은 가장 가까운 방향까지 중앙값 22°입니다(`03` §4).	내 판단
DexTrack, PHC류 다중 모션 추적	시퀀스별 정책을 증류하거나 하나로 합치는 계보입니다. CHORD 4,739개를 데이터로 쓰려면 결국 이 단계가 필요합니다.	GNR 계보도(`gnr/02` §1)에도 있음

7. GitHub 정리 (2026-11-05 `gh api` 확인)

저장소	무엇인가	공식 여부	라이선스	★	마지막 push	확인 커밋
nvidia-isaac/video_to_data	CHORD 코드(`robotic_grounding/`: Isaac Lab PPO 경로 + `flash_chord/` FlashSAC 경로), 영상 복원, GR00T 후속 학습	**공식**	코드 Apache-2.0 + 문서 CC BY 4.0 (GitHub 표시는 NOASSERTION)	943	2026-11-02	`9cad120` (`robotic_grounding/`만 sparse clone, 미디어 제외)
Hugging Face `nvidia/video-to-data-robot-dexterity-task-library-and-dataset`	4,739개 과제 라이브러리와 LeRobot 형식 로봇 데이터	공식 데이터	CC BY 4.0	—	2026-08-21	`2376c2e` (메타데이터만, 파일 목록 API 상한 10만 개)
MandiZhao/dexmachina	DexMachina [1] 기준선	기준선 공식	MIT	280	2026-03-23	`adae5bf` (메타데이터만)
ManipTrans/ManipTrans	ManipTrans [2] 기준선	기준선 공식	GPL-3.0	349	2025-10-10	`a3d08cf` (메타데이터만)
facebookresearch/spider	SPIDER [3] 기준선, SP-SR 스크립트	기준선 공식	CC BY-NC (LICENSE 본문, GitHub 표시 NOASSERTION)	595	2026-09-22	`4471700` (`flashdexretarget/`에서 지표 스크립트 확인)

저장소	무엇인가	공식 여부	라이선스	★	마지막 push	확인 커밋
tylerlum/human2sim2robot	Human2Sim2Robot [43] 기준선	기준선 공식	MIT	161	2026-08-27	`894eae2` (메타데이터만)
Holiday-Robot/FlashSAC	FlashSAC 학습기 (FlashCHORD가 쓰는 알고리즘)	구성요소	MIT	505	2026-04-09	`87edc90` (메타데이터만)
Holiday-Robot/FlashDexRetarget	CHORD를 기준선으로 쓴 후속 연구	관련 연구 공식	MIT	70	2026-10-20	`6f95e13` (`flashdexretarget/` 에서 대조)
leggedrobotics/rsl_rl	PPO 구현 (Isaac Lab 경로)	구성요소	BSD-3 계열 (LICENSE 본문, GitHub 표시 NOASSERTION)	3,119	2026-10-28	`1156331` (메타데이터만)
isaac-sim/IsaacLab	시뮬레이션 프레임워크 [42]	구성요소	BSD-3-Clause (LICENSE 본문, GitHub 표시 NOASSERTION)	8,569	2026-11-05	`f644e1f` (메타데이터만)
newton-physics/newton	FlashCHORD의 물리 엔진	구성요소	Apache-2.0	5,887	2026-11-05	`1abe073` (메타데이터만)
NVlabs/GR00T-WholeBodyControl	SONIC [20] 전신 제어기	구성요소 (같은 그룹)	코드 Apache-2.0 + 가중치 NVIDIA Open Model License	3,883	2026-11-03	`1e1ec4f` (메타데이터만)
NVlabs/motionbricks	MotionBricks [37] inpainting 구조	구성요소 (같은 그룹)	코드 Apache-2.0 + 문서·자산 CC BY 4.0	905	2026-10-26	`54fc563` (메타데이터만)

- video_to_data의 LICENSE 첫 줄은 "Copyright (c) 2026 NVIDIA CORPORATION & AFFILIATES. All rights reserved."이고, 이어서 "문서·skill은 CC BY 4.0, 소스 코드는 Apache-2.0의 이중 라이선스"라고 적었습니다. 코드 자체는 쓸 수 있습니다. MANO와 원천 데이터셋(ARCTIC, HOT3D 등)은 각자의 라이선스를 따로 받아야 합니다 (`robotic_grounding/README.md`).
- 저장소 이력: 첫 커밋 2026-06-23(arXiv v1 다음 날), 학습 코드 공개 v0.2.0 `cbee067` (2026-08-14, arXiv v2와 같은 날), v0.3.0 `dd395a7` (2026-09-17), FlashCHORD가 처음 들어온 v0.4.0 `33129dd` (2026-09-23), 그 뒤 `aee95f2` (09-28), `8c778f9` (09-29)에서 FlashCHORD를 다듬었습니다.